VOLUME 5

SQL & Database Internals

Query writing, indexes, execution plans, MVCC, isolation, locking, PostgreSQL, caching, Redis

Java 21 · Spring Boot 3 · SQL · Microservices · AI integration
Interview preparation for 12+ years experience · real scenarios from the RefinePoint platform

Contents

23. SQL: From Fundamentals to Advanced Queries

23.1 SQL categories
23.2 Logical query processing order
23.3 Joins
23.4 NULL and three-valued logic
23.5 Grouping, subqueries and CTEs
23.6 Window functions
23.7 Set operations and useful expressions
23.8 UPSERT
23.9 DELETE vs TRUNCATE vs DROP
23.10 Keys and constraints
23.11 Normalization
23.12 Views, procedures, triggers, JSONB
23.13 Interview questions and answers
23.14 Classic SQL problems
23.15 Reconciliation-flavoured queries

24. Database Internals & Performance (PostgreSQL focus)

24.1 How PostgreSQL stores a table
24.2 B+tree indexes — the default and why
24.3 Composite, covering, partial and expression indexes
24.4 Other index types
24.5 When an index is NOT used
24.6 Reading EXPLAIN and EXPLAIN ANALYZE
24.7 Statistics and ANALYZE
24.8 MVCC internals
24.9 WAL, durability and checkpoints
24.10 Locking
24.11 Isolation levels and how Postgres implements them
24.12 Connection pooling
24.13 N+1 at the database level
24.14 Slow-query troubleshooting playbook
24.15 Partitioning
24.16 Replication and read replicas
24.17 Sharding, consistent hashing, CAP and PACELC
24.18 OLTP vs OLAP: row storage vs columnar storage
24.19 Schema migrations with Flyway, without downtime
24.20 Multi-tenant indexing
24.21 Interview questions and answers

25. Caching, Redis & NoSQL

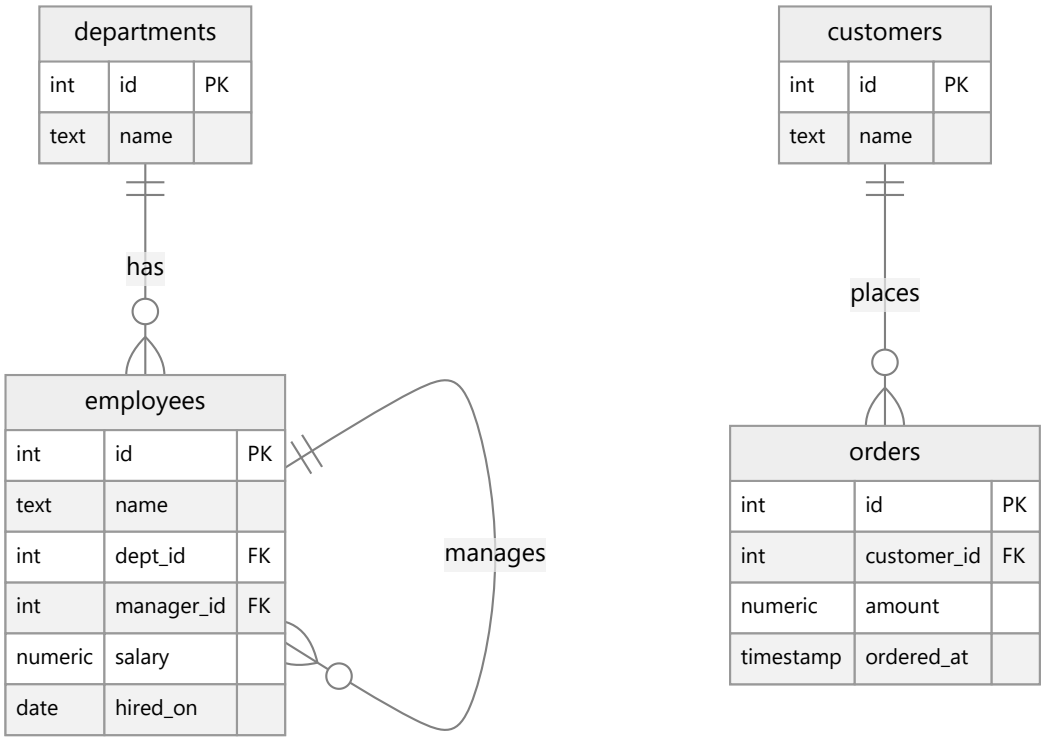
25.1 Why cache, and where
25.2 Caching patterns
25.3 Spring Cache abstraction

25.4 Local (Caffeine) vs distributed (Redis)
25.5 Invalidation and consistency
25.6 Failure modes: stampede, penetration, avalanche, hot keys
25.7 Redis internals
25.8 Persistence, replication and clustering
25.9 Distributed locks
25.10 Rate limiting
25.11 Sessions and pub/sub
25.12 NoSQL families
25.13 Interview questions and answers

23. SQL: From Fundamentals to Advanced Queries

Twelve years in, you will still be handed a whiteboard and asked for the department-wise top three earners. Interviewers use SQL to test precise thinking: do you know why `NOT IN` returned nothing, why your alias is not visible in `WHERE`, why `LAST_VALUE` gave the current row? This chapter explains the logic first, then works through more than forty classic problems. Everything is PostgreSQL unless noted, and we finish with reconciliation queries of the kind our product runs.

The examples use this schema:



23.1 SQL categories

Category	Purpose	Statements
DDL (definition)	Structure	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML (manipulation)	Rows	`INSERT`, `UPDATE`, `DELETE`, `MERGE` (and `SELECT`, sometimes split out as DQL)
DCL (control)	Permissions	`GRANT`, `REVOKE`
TCL (transaction)	Transaction boundaries	`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`

DDL is transactional in PostgreSQL: you can `BEGIN; ALTER TABLE ...; ROLLBACK;`. Oracle and MySQL commit implicitly around DDL. That is why Flyway migrations are safe to run in a transaction on PostgreSQL.

23.2 Logical query processing order

SQL is written in one order and evaluated logically in another. This single diagram answers a dozen interview questions.



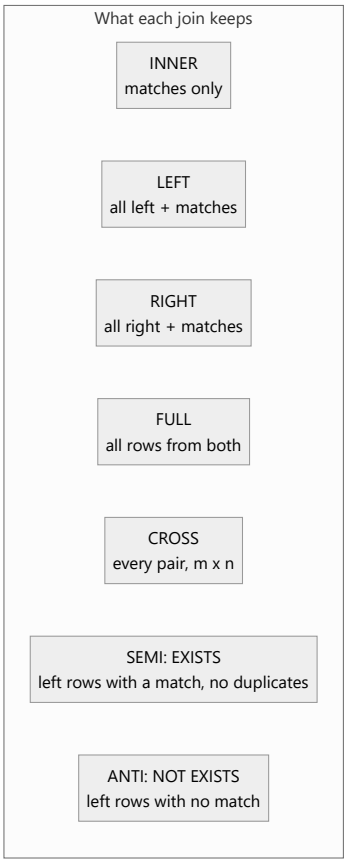
Consequences:

- `WHERE` cannot see `SELECT` aliases or aggregates, as they do not exist yet. `ORDER BY` can see aliases.
- Aggregates belong in `HAVING`; row filters belong in `WHERE` (it is cheaper to discard rows before grouping).
- Window functions cannot appear in `WHERE` or `HAVING`. Wrap them in a subquery or CTE and filter outside.
- The planner may physically reorder anything, as long as the result is the same. This order describes meaning, not execution.

23.3 Joins

```
employees                departments
id name   dept_id         id name
1  Asha   10              10 Finance
2  Ben    20              20 Ops
3  Chen   NULL            30 Legal

INNER JOIN       LEFT JOIN          RIGHT JOIN          FULL OUTER JOIN
Asha  Finance    Asha  Finance      Asha  Finance       Asha  Finance
Ben   Ops        Ben   Ops          Ben   Ops           Ben   Ops
                 Chen  NULL         NULL  Legal         Chen  NULL
                                                      NULL  Legal
```



Self join: a table joined to itself under two aliases, as in employees with their managers. **Cross join:** the Cartesian product, useful with a calendar or `generate_series` to fill gaps. **`LATERAL`:** a subquery in `FROM` that can reference earlier tables, like a per-row function call.

> **Trap:** a filter on the right table in `WHERE` undoes a LEFT JOIN. `LEFT JOIN orders o ON o.customer_id = c.id WHERE o.amount > 100` drops customers with no orders, because `NULL > 100` is not true. Put the condition in `ON` to keep them.

ON vs WHERE

For inner joins they are equivalent. For outer joins, `ON` decides what matches, and unmatched left rows survive with NULLs; `WHERE` then filters the result.

Semi and anti joins

```sql
-- Semi: customers with at least one order. No duplicates even with many orders.
SELECT c.* FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- Anti: customers with no orders.
SELECT c.* FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

> **Trap:** `WHERE id NOT IN (SELECT customer_id FROM orders)` returns **zero rows** if any `customer_id` is NULL. `x NOT IN (1, NULL)` means `x <> 1 AND x <> NULL`, and `x <> NULL` is UNKNOWN, so the whole condition is never TRUE. `NOT EXISTS` is immune and usually plans better as an anti join. Use it always.

23.4 NULL and three-valued logic

NULL means "unknown", so comparisons return UNKNOWN rather than true or false. `WHERE` keeps only TRUE.

Expression	Result
`NULL = NULL`	UNKNOWN (use `IS NULL`)
`NULL AND FALSE`	FALSE
`NULL AND TRUE`	UNKNOWN
`NULL OR TRUE`	TRUE
`NOT NULL`	UNKNOWN
`1 + NULL`, `'a' || NULL`	NULL

More rules: aggregates ignore NULLs except `COUNT(*)`; `SUM` over no rows is NULL (`COALESCE(SUM(x), 0)`); `GROUP BY` and `DISTINCT` treat NULLs as one group; `UNIQUE` allows many NULLs unless declared `NULLS NOT DISTINCT` (PostgreSQL 15). PostgreSQL sorts NULLs last ascending and first descending; override with `NULLS FIRST/LAST`. `IS DISTINCT FROM` is a NULL-safe comparison, ideal for "did this value change?".

23.5 Grouping, subqueries and CTEs

```sql
SELECT dept_id, count(*) AS people, avg(salary) AS avg_sal,
       count(*) FILTER (WHERE salary > 100000) AS high_earners
FROM employees
WHERE hired_on >= DATE '2020-01-01'
GROUP BY dept_id
HAVING count(*) >= 5;
```

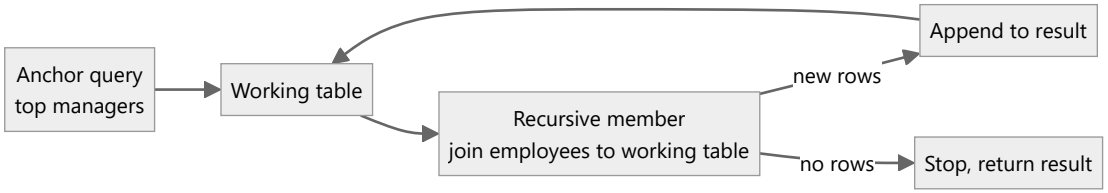
Every selected column must be grouped or aggregated (or functionally dependent on a grouped primary key). `ROLLUP`, `CUBE` and `GROUPING SETS` add subtotals.

Subqueries: a *non-correlated* subquery runs logically once; a *correlated* one references the outer row and runs logically per row (`WHERE salary > (SELECT avg(salary) FROM employees x WHERE x.dept_id = e.dept_id)`). The planner often flattens both into joins.

CTEs (`WITH`) name a subquery for readability. Since PostgreSQL 12, a side-effect-free CTE referenced once is inlined; earlier it was always materialized, which acted as an optimisation fence. Control it with `AS MATERIALIZED` / `AS NOT MATERIALIZED`.

Recursive CTEs

```sql
WITH RECURSIVE tree AS (
    SELECT id, name, manager_id, 1 AS depth, name::text AS path
    FROM employees WHERE manager_id IS NULL             -- anchor
  UNION ALL
    SELECT e.id, e.name, e.manager_id, t.depth + 1, t.path || ' > ' || e.name
    FROM employees e JOIN tree t ON e.manager_id = t.id -- recursive member
)
SELECT * FROM tree ORDER BY path;
```



Guard against cycles: PostgreSQL 14 has `CYCLE id SET is_cycle USING path`; earlier, keep an array of visited ids and test `e.id <> ALL(path)`. Using `UNION` rather than `UNION ALL` also removes duplicates at each step.

23.6 Window functions

A window function computes across related rows **without collapsing them**, as `GROUP BY` would.

```
SELECT name, dept_id, salary,
       ROW_NUMBER() OVER w AS rn,
       RANK()       OVER w AS rnk,
       DENSE_RANK() OVER w AS drnk,
       LAG(salary)  OVER w AS prev_salary,
       SUM(salary)  OVER (PARTITION BY dept_id) AS dept_total
FROM employees
WINDOW w AS (PARTITION BY dept_id ORDER BY salary DESC);
```

salary	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

Frames choose which rows the function sees relative to the current one.

```
ROWS  BETWEEN 2 PRECEDING AND CURRENT ROW   → physical rows: 3-row moving window
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW → by value: includes all PEERS (ties) of the
current row
GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW  → by peer groups

date        amount   SUM ... ROWS     SUM ... RANGE (the default with ORDER BY)
2024-01-01   10       10               10
2024-01-02   20       30               60    ← both Jan 2 rows are peers
2024-01-02   30       60               60
```

Trap: with `ORDER BY` and no frame, the default is `RANGE UNBOUNDED PRECEDING TO CURRENT ROW`. So running totals jump across ties, and `LAST_VALUE(x)` returns the current row's last peer rather than the partition's last row. Write `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` for `LAST_VALUE`, and `ROWS` for running totals.

23.7 Set operations and useful expressions

`UNION` removes duplicates (sort or hash cost); `UNION ALL` does not, so prefer it when duplicates are impossible or wanted. `INTERSECT` and `EXCEPT` compare whole rows and treat NULLs as equal, unlike joins. Columns match by position, not name.

```
SELECT CASE WHEN amount < 0 THEN 'DEBIT' ELSE 'CREDIT' END,
       COALESCE(nickname, name),           -- first non-null
       NULLIF(total, 0),                   -- NULL if zero: avoids division by zero
       upper(trim(reference)), split_part(reference, '-', 2), length(name),
       regexp_replace(reference, '[^0-9]', '', 'g'),
       string_agg(name, ', ' ORDER BY name),
       date_trunc('month', ordered_at), ordered_at::date,
       EXTRACT(DOW FROM ordered_at), age(now(), hired_on),
       now() - interval '30 days', CURRENT_DATE - hired_on   -- date - date = integer days
FROM ...;
```

String and date functions you should write without looking anything up

Task	PostgreSQL	Note
Concatenate	`a	
Substring	`substring(s from 1 for 3)`, `left(s, 3)`	Positions start at 1
Replace, pattern	`replace`, `regexp_replace`, `~`, `ILIKE`	A leading `%` in `LIKE` cannot use a plain B-tree index
Split, aggregate	`split_part`, `string_to_array`, `string_agg`	`string_agg` is the inverse of a split
Convert	`to_char(d, 'YYYY-MM')`, `to_date`, `::numeric`	Casting bad data aborts the whole statement
Bucket a date	`date_trunc('month', t)`	Returns a timestamp, so it sorts correctly, unlike `to_char`
Date arithmetic	`d + 1`, `d2 - d1`, `t + interval '1 month'`	Date minus date is integer days; timestamp minus timestamp is an interval
Parts	`EXTRACT(YEAR FROM t)`, `age(a, b)`	`DOW` counts Sunday as 0, and `ISODOW` counts Monday as 1

23.8 UPSERT

```
INSERT INTO balances (org_id, account, amount) VALUES (?, ?, ?)
ON CONFLICT (org_id, account) DO UPDATE
   SET amount = EXCLUDED.amount, updated_at = now()
   WHERE balances.amount IS DISTINCT FROM EXCLUDED.amount;

INSERT INTO consumed_events (consumer, event_id) VALUES (?, ?) ON CONFLICT DO NOTHING;
```

The `WHERE` skips no-op updates, which would otherwise write a new row version for nothing. The statement is atomic even under concurrency, unlike check-then-insert, thanks to the unique index. `EXCLUDED` is the row proposed for insertion. `MERGE` (PostgreSQL 15) is the standard form but is not race-free in the same way: it can raise unique violations under concurrency.

> **RefinePoint:** `ConsumedEvents.firstTime(consumer, eventId)` runs the second statement. If `update` returns 1, this consumer has never seen the event. Because it runs in the handler's transaction, Kafka's at-least-once redelivery becomes harmless. `RunWorker` heartbeats use the `DO UPDATE` form on `run_workers`.

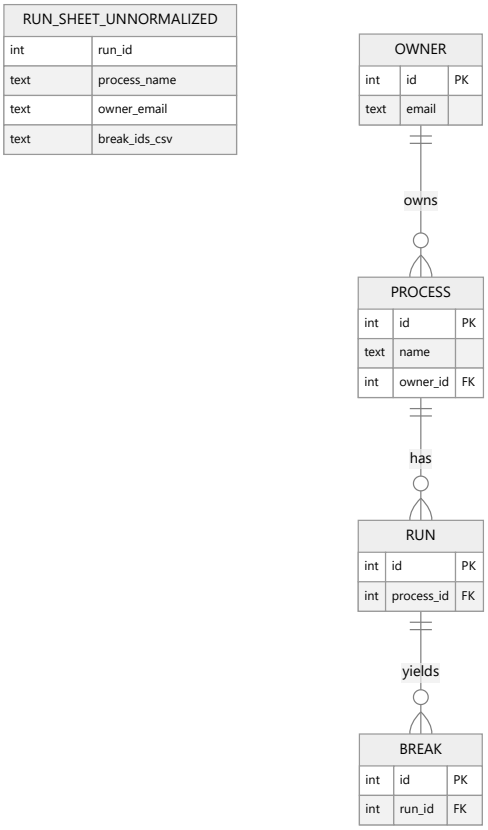
23.9 DELETE vs TRUNCATE vs DROP

	DELETE	TRUNCATE	DROP
Removes	Selected rows	All rows	Table and definition
`WHERE`	Yes	No	No
Row triggers	Fire	Do not fire	No
Speed	Row by row, leaves dead tuples	Fast, new file	Fast
Lock	Row locks	`ACCESS EXCLUSIVE`	`ACCESS EXCLUSIVE`
Rollback	Yes	Yes in PostgreSQL; implicit commit in Oracle/MySQL	Yes in PostgreSQL
Identity	Continues	`RESTART IDENTITY` optional	Gone
Foreign keys	Checked per row	Refused if referenced, unless `CASCADE`	Refused unless `CASCADE`

23.10 Keys and constraints

Primary key (unique + not null, one per table), foreign key (`ON DELETE CASCADE / SET NULL / RESTRICT`; PostgreSQL does **not** index FK columns automatically, so add one), unique, `NOT NULL`, `CHECK`, `DEFAULT`, and PostgreSQL's `EXCLUDE` (no overlapping date ranges). Candidate key: any minimal unique column set. Natural vs surrogate key: natural keys carry meaning but change; surrogates (`bigint`, UUID) are stable. Random UUID v4 hurts B-tree locality; UUID v7 is time-ordered.

23.11 Normalization



Form	Rule	Violation	Fix
1NF	Atomic values, no repeating groups	`break_ids = '1,2,3'`	One row per break
2NF	1NF + no non-key column depends on **part** of a composite key	`(order_id, product_id) → product_name`	Move to `products`
3NF	2NF + no non-key depends on another non-key	`process_id → owner_id → owner_email`	Move to `owners`
BCNF	Every determinant is a superkey	`(student, course) → teacher`, with `teacher → course`	Split into `(teacher, course)` and `(student, teacher)`

Normalization removes update, insert and delete anomalies. **Denormalize deliberately** when read performance demands it, and name your synchronization mechanism: a materialized view, a summary table, or a snapshot taken for history (an invoice copies the price because it must not change later). Reporting schemas (star) are denormalized by design.

> **RefinePoint:** `org_id` is repeated on every tenant table even where a join could derive it. Strictly that is redundant, but it lets every query filter by tenant without a join and makes the isolation rule trivial to check.

23.12 Views, procedures, triggers, JSONB

View: a stored query, expanded at plan time, holding no data; simple views are updatable. **Materialized view:** stores results and goes stale until `REFRESH MATERIALIZED VIEW [CONCURRENTLY]`; concurrent refresh needs a unique index and does not block readers.

Functions and procedures: a function returns a value, can appear in a query, and runs inside the caller's transaction. A procedure (`CALL`, PostgreSQL 11) can `COMMIT` inside. **Triggers** run automatically on data changes.

For	Against
Close to the data, fewer round trips	Hidden logic that app developers miss
Enforce rules for every client	Harder to version, test and debug
Set-based bulk work	Harder to scale; ties you to one vendor

Reasonable uses: `updated_at` triggers, audit tables, partition routing, heavy set-based operations. Business rules belong in the application in most Java shops.

JSONB stores binary parsed JSON, and supports indexing.

```
SELECT details->>'reason' FROM audit WHERE details @> '{"action":"APPROVE"}';   -- ->> gives
text, -> gives jsonb
CREATE INDEX ON audit USING gin (details jsonb_path_ops);                        -- speeds up @>
UPDATE settings SET data = jsonb_set(data, '{ai,enabled}', 'true') WHERE org_id = ?;
```

Use it for flexible attributes and payloads (such as event envelopes). Do not use it for fields you join, constrain or update often: statistics are weak, and each update rewrites the entire value.

23.13 Interview questions and answers

Q1. What is the logical execution order of a SELECT?

FROM/JOIN, WHERE, GROUP BY, HAVING, window functions, SELECT, DISTINCT, set operations, ORDER BY, LIMIT. It explains alias visibility and why window functions cannot go in WHERE. The optimizer can execute differently if results match.

Q2. WHERE vs HAVING?

`WHERE` filters rows before grouping; `HAVING` filters groups afterward and can use aggregates. Put non-aggregate conditions in `WHERE`: it discards rows earlier, so it is faster. `HAVING` without `GROUP BY` treats the whole table as one group.

Q3. Why does this fail: `SELECT salary*12 AS annual FROM employees WHERE annual > 100000`?

`WHERE` runs before `SELECT`, so the alias does not exist. Repeat the expression or use a subquery or CTE. `ORDER BY annual` would work.

Q4. Predict the counts: table `t(x)` holds 1, 2, NULL.

```
SELECT count(*), count(x), count(DISTINCT x), sum(x), avg(x) FROM t;  -- 3, 2, 2, 3, 1.5
SELECT * FROM t WHERE x <> 1;                                        -- only 2; NULL is excluded
SELECT count(*) FROM t WHERE x IS NULL;                              -- 1
```

`avg` divides by 2, not 3. To treat NULL as zero use `avg(COALESCE(x, 0))`.

Q5. Why can `NOT IN` return nothing, and what do you use?

A NULL in the list makes `NOT IN` UNKNOWN for every row. Use `NOT EXISTS`, which correlates per row and has true anti-join semantics. It also has better plans.

Q6. EXISTS vs IN vs JOIN for a semi-join?

`EXISTS` and `IN` give the same plan in modern PostgreSQL for positive semi-joins, and neither duplicates outer rows. `JOIN` duplicates a row per match, leading people to add `DISTINCT` at extra cost. Use `EXISTS` when you only test presence.

Q7. UNION vs UNION ALL?

`UNION` deduplicates with a sort or hash; `UNION ALL` concatenates. Use `UNION ALL` by default. Also, each side's `ORDER BY` needs parentheses; a final `ORDER BY` applies to the whole.

Q8. RANK vs DENSE_RANK vs ROW_NUMBER?

On ties: `ROW_NUMBER` is arbitrary and unique (add a tiebreak for determinism), `RANK` repeats and leaves gaps (1,2,2,4), `DENSE_RANK` repeats with no gaps (1,2,2,3). "Top 3 salaries" means `DENSE_RANK`; "top 3 people" usually `ROW_NUMBER`. Ask which.

Q9. ROWS vs RANGE?

`ROWS` counts physical rows; `RANGE` uses the `ORDER BY` value and includes all ties (peers). The default with `ORDER BY` is `RANGE`, a source of surprising running totals and `LAST_VALUE`. Also `RANGE` allows value offsets: `RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW`.

Q10. GROUP BY vs window function?

`GROUP BY` collapses to one row per group. Windows keep every row and add a computed column. Use a window for "each employee with their department's average", where grouping alone would need a self-join.

Q11. CTE vs subquery vs temp table?

CTE: readable, query-scoped, usually inlined since PostgreSQL 12. Subquery: same effect, harder to read. Temp table: session-scoped, stored, can be indexed and `ANALYZE`d, so best for huge multi-step work.

Q12. How do you get back the rows a statement changed?

`RETURNING` returns changed rows in one round trip, so no separate select: `UPDATE recon_runs SET instance = NULL WHERE ... RETURNING id`. It also works on `INSERT` and `DELETE`.

Q13. Can `DISTINCT` apply to one column?

No: `SELECT DISTINCT a, b` is about the pair. PostgreSQL has `DISTINCT ON (a)` for "first row per a by the `ORDER BY`": `SELECT DISTINCT ON (customer_id) * FROM orders ORDER BY customer_id, ordered_at DESC`.

Q14. Is `COUNT(1)` faster than `COUNT(*)`?

No. PostgreSQL's `count(*)` is the optimised one, and `count(1)` evaluates a constant per row, marginally slower. `COUNT(col)` differs in meaning: it skips NULLs.

Q15. What makes a query non-SARGable, and why does it matter?

Wrapping an indexed column in a function or expression prevents plain index use. `WHERE date(ordered_at) = '2024-01-01'` scans; `WHERE ordered_at >= '2024-01-01' AND ordered_at < '2024-01-02'` uses the index. Avoid `BETWEEN` for timestamps, since it includes the end instant. Details in chapter 24.

Q16. OFFSET pagination vs keyset?

`OFFSET 100000` reads and discards 100,000 rows, and results shift if rows are inserted. Keyset is `WHERE (created_at, id) < (?, ?) ORDER BY created_at DESC, id DESC LIMIT 50`: constant cost on an index and stable pages, but no jumping to page N.

Q17. Primary key vs unique key?

Primary key: one per table, implies `NOT NULL`, the default FK target. Unique: many per table, allows NULLs by default. Both build a unique B-tree index in PostgreSQL.

Q18. CHAR, VARCHAR or TEXT in PostgreSQL?

`TEXT` and `VARCHAR` perform identically; `VARCHAR(n)` only adds a length check. `CHAR(n)` pads and is rarely wanted. Use `NUMERIC` for money, never `FLOAT`. Use `TIMESTAMPTZ` for instants.

Q19. What is the difference between a function and a procedure?

A function returns a value and can sit inside a query, running in the caller's transaction. A procedure is invoked by `CALL` and may commit or roll back internally, useful for batch maintenance.

Q20. Should you use triggers?

Sparingly: `updated_at`, audit capture, invariants that every client must obey. Business logic in triggers is invisible to the Java developer reading the service, is hard to test, and cascades unpredictably.

Q21. Predict the result: count of orders per customer, with a LEFT JOIN.

```
SELECT c.id, count(*) AS wrong, count(o.id) AS right_count
FROM customers c LEFT JOIN orders o ON o.customer_id = c.id GROUP BY c.id;
```

A customer with no orders shows `wrong = 1` and `right_count = 0`. The outer join produces one row with NULLs on the order side; `count(*)` counts that row, while `count(o.id)` skips the NULL.

Q22. What do aggregates return on an empty set?

`SELECT count(*), sum(x) FROM t WHERE false` returns **one row**: `0, NULL`. Add `GROUP BY y` and it returns **zero rows**, as there are no groups. So a report that expects zeros per category must start from the category table and left join to the data, using `COALESCE(sum(x), 0)`.

Q23. Why are my totals doubled after adding a join?

Fan-out. Joining a customer to 3 orders and 2 payments yields 6 rows, so `sum(o.amount)` counts each order twice. Aggregate each child table separately, then join the results, or use correlated subqueries. `count(DISTINCT ...)` hides the row count, but it cannot fix a `SUM`. This is the most common bug in hand-written reporting SQL.

Q24. What do ROLLUP, CUBE and GROUPING SETS do?

They produce several levels of grouping from one query.

```
SELECT dept_id, EXTRACT(YEAR FROM hired_on) AS yr, sum(salary), GROUPING(dept_id) AS is_total
FROM employees GROUP BY ROLLUP (dept_id, yr);
```

`ROLLUP (a, b)` gives `(a, b)`, `(a)` and the grand total. `CUBE` gives every combination. `GROUPING SETS` lists exactly what you want. `GROUPING()` distinguishes a subtotal's NULL from a real NULL in the data.

Q25. How do you check that two tables hold identical data?

```
(SELECT * FROM a EXCEPT ALL SELECT * FROM b)
UNION ALL
(SELECT * FROM b EXCEPT ALL SELECT * FROM a);       -- no rows means identical
```

`EXCEPT` treats NULLs as equal, so this avoids the `col = col` problem of a join, and `ALL` respects duplicates. Plain `EXCEPT` would treat a row present once and a row present twice as the same. It is a handy test after a migration.

Q26. Without ORDER BY, is the order of rows guaranteed?

No, never. A heap scan usually looks ordered until a vacuum, an update, a parallel plan or a synchronized scan changes it. Equally, `ORDER BY` in a subquery does not guarantee the outer order. `LIMIT` without `ORDER BY` returns an arbitrary row, and an `ORDER BY` on a non-unique column makes pages unstable, so add a unique tiebreak.

Q27. TIMESTAMP vs TIMESTAMPTZ, and JSON vs JSONB?

`TIMESTAMPTZ` stores an instant (normalised to UTC) and converts to the session zone on display. `TIMESTAMP` stores wall-clock digits with no zone, so use it only for things like "9 am local, wherever". A day-bucket report depends on the session zone, so state the zone explicitly: `date_trunc('day', t, 'Asia/Kolkata')` in PostgreSQL 12+. `JSON` stores exact text and reparses on each read; `JSONB` stores a parsed binary form, drops duplicate keys and key order, and supports GIN indexes. Choose `JSONB` unless you must keep the original text exactly.

23.14 Classic SQL problems

Q28. Nth highest salary, four ways.

```sql
-- 1. DENSE_RANK (best: handles ties, any N)
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS r FROM employees) x WHERE r = 3;
-- 2. OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2;
-- 3. Correlated count
SELECT DISTINCT salary FROM employees e
WHERE 2 = (SELECT count(DISTINCT salary) FROM employees x WHERE x.salary > e.salary);
-- 4. Second highest only
SELECT max(salary) FROM employees WHERE salary < (SELECT max(salary) FROM employees);
```

Say what happens when no Nth value exists: empty set for 1–3, NULL for 4.

Q29. Nth highest salary per department.

```sql
SELECT dept_id, name, salary FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r FROM employees) x
WHERE r = 2;
```

Q30. Department-wise top 3 earners, with department name.

```sql
WITH ranked AS (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r FROM employees
e)
SELECT d.name, r.name, r.salary FROM ranked r JOIN departments d ON d.id = r.dept_id
WHERE r.r <= 3 ORDER BY d.name, r.salary DESC;
```

Q31. Find duplicate rows.

```sql
SELECT email, count(*) FROM users GROUP BY email HAVING count(*) > 1;
```

Q32. Delete duplicates, keeping the lowest id.

```
DELETE FROM users WHERE id IN (
  SELECT id FROM (SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn FROM users)
x
  WHERE rn > 1);
-- no primary key? PostgreSQL's physical row id works:
DELETE FROM users a USING users b WHERE a.ctid > b.ctid AND a.email = b.email;
```

Afterward add a `UNIQUE` constraint, or duplicates return.

Q33. Employees who earn more than their manager.

```
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id WHERE e.salary > m.salary;
```

Q34. Employees earning above their department's average.

```
SELECT name, salary FROM (
  SELECT *, avg(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM employees) x
WHERE salary > dept_avg;
```

Q35. Employee with the highest salary in each department (all ties).

```
SELECT dept_id, name, salary FROM employees e
WHERE salary = (SELECT max(salary) FROM employees x WHERE x.dept_id = e.dept_id);
```

Q36. Departments with no employees.

```
SELECT d.name FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.id);
```

Q37. Customers who never ordered.

```
SELECT c.name FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;
```

Test a NOT NULL column of the right table (`o.id`), not a nullable one. `NOT EXISTS` is equivalent.

Q38. Second most recent order for each customer.

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY ordered_at DESC, id DESC) AS
rn FROM orders o) x
WHERE rn = 2;
```

Tiebreak on `id` to be deterministic.

Q39. Latest order per customer, three ways.

```
SELECT DISTINCT ON (customer_id) * FROM orders ORDER BY customer_id, ordered_at DESC;   -- concise
SELECT c.id, l.* FROM customers c
CROSS JOIN LATERAL (SELECT * FROM orders o WHERE o.customer_id = c.id
                    ORDER BY ordered_at DESC LIMIT 1) l;                                -- fast with an index
-- or ROW_NUMBER as in the previous question
```

Q40. Running total per day.

```
SELECT day, total, SUM(total) OVER (ORDER BY day ROWS UNBOUNDED PRECEDING) AS running
FROM (SELECT ordered_at::date AS day, sum(amount) AS total FROM orders GROUP BY 1) d;
```

Q41. Seven-day moving average.

```
SELECT day, avg(total) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) FROM daily;
```

`ROWS` assumes one row per day; with missing days, use `RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW`, or fill the calendar first.

Q42. Month-over-month growth percentage.

```
WITH m AS (
  SELECT date_trunc('month', ordered_at) AS month, sum(amount) AS rev FROM orders GROUP BY 1)
SELECT month, rev,
       round(100.0 * (rev - LAG(rev) OVER (ORDER BY month)) / NULLIF(LAG(rev) OVER (ORDER BY month), 0), 2) AS pct
FROM m ORDER BY month;
```

`100.0` avoids integer division, and `NULLIF` avoids division by zero. Missing months give the wrong "previous month", so fill from `generate_series` if needed.

Q43. Consecutive login days (gaps and islands).

```
WITH d AS (SELECT DISTINCT user_id, login_at::date AS day FROM logins),
     g AS (SELECT user_id, day, day - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY day))::int AS island FROM d)
SELECT user_id, min(day) AS from_day, max(day) AS to_day, count(*) AS streak
FROM g GROUP BY user_id, island HAVING count(*) >= 3;
```

```
day         row_number   day - row_number
Jan 10      1            Jan 9    ┐ island 1: the difference stays constant
Jan 11      2            Jan 9    ┘ while days are consecutive
Jan 14      3            Jan 11   ┐ island 2: a gap shifts the difference
Jan 15      4            Jan 11   ┘
```

Q44. Find missing dates or ids (gaps).

```
SELECT d::date FROM generate_series(DATE '2024-01-01', DATE '2024-01-31', interval '1 day') d
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.ordered_at::date = d::date);
-- gaps in ids
SELECT id + 1 AS gap_from, next_id - 1 AS gap_to
FROM (SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id FROM invoices) x WHERE next_id > id + 1;
```

Q45. Pivot: revenue per customer with a column per quarter.

```
SELECT customer_id,
       sum(amount) FILTER (WHERE EXTRACT(QUARTER FROM ordered_at) = 1) AS q1,
       sum(amount) FILTER (WHERE EXTRACT(QUARTER FROM ordered_at) = 2) AS q2,
       sum(amount) FILTER (WHERE EXTRACT(QUARTER FROM ordered_at) = 3) AS q3,
       sum(amount) FILTER (WHERE EXTRACT(QUARTER FROM ordered_at) = 4) AS q4
FROM orders GROUP BY customer_id;
```

On other databases write `SUM(CASE WHEN ... THEN amount END)`. `crosstab` (tablefunc) handles dynamic columns, though the column list must still be declared.

Q46. Unpivot columns into rows.

```
SELECT id, v.quarter, v.amount
FROM sales s CROSS JOIN LATERAL (VALUES ('q1', s.q1), ('q2', s.q2), ('q3', s.q3), ('q4', s.q4))
v(quarter, amount);
```

Q47. Median salary.

```
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY salary) FROM employees;  -- interpolates
SELECT dept_id, percentile_disc(0.5) WITHIN GROUP (ORDER BY salary) FROM employees GROUP BY
dept_id; -- an actual value
-- without percentile functions
SELECT avg(salary) FROM (SELECT salary, ROW_NUMBER() OVER (ORDER BY salary) r, count(*) OVER () c
FROM employees) x
WHERE r IN ((c + 1) / 2, (c + 2) / 2);
```

Q48. Top N percent of customers by revenue.

```
SELECT * FROM (SELECT customer_id, sum(amount) AS rev,
                      NTILE(10) OVER (ORDER BY sum(amount) DESC) AS decile
               FROM orders GROUP BY customer_id) x WHERE decile = 1;
```

Note that a window function can wrap an aggregate here: windows are evaluated after `GROUP BY`.

Q49. Customers who bought every product (relational division).

```
SELECT customer_id FROM order_items GROUP BY customer_id
HAVING count(DISTINCT product_id) = (SELECT count(*) FROM products);
```

Q50. Customers who bought A but not B.

```
SELECT customer_id FROM order_items GROUP BY customer_id
HAVING bool_or(product = 'A') AND NOT bool_or(product = 'B');
```

Q51. First purchase per customer and cohort retention.

```
WITH first AS (SELECT customer_id, date_trunc('month', min(ordered_at)) AS cohort FROM orders
GROUP BY 1)
SELECT f.cohort, date_trunc('month', o.ordered_at) AS month, count(DISTINCT o.customer_id) AS
active
FROM orders o JOIN first f USING (customer_id) GROUP BY 1, 2 ORDER BY 1, 2;
```

Q52. Swap M and F with one update, and swap two columns.

```
UPDATE employees SET gender = CASE gender WHEN 'M' THEN 'F' ELSE 'M' END;
UPDATE t SET a = b, b = a;   -- right-hand sides see the OLD row in PostgreSQL
```

MySQL evaluates left to right, so its `SET a = b, b = a` sets both to `b`. A good portability catch.

Q53. Rows with a value greater than the previous row (rising temperature).

```
SELECT id FROM (SELECT id, temp, day, LAG(temp) OVER (ORDER BY day) AS p, LAG(day) OVER (ORDER BY
day) AS pd FROM weather) x
WHERE temp > p AND day = pd + 1;
```

The `day` check guards against gaps.

Q54. Org hierarchy: everyone under a manager, with depth.

```
WITH RECURSIVE sub AS (
  SELECT id, name, 0 AS depth FROM employees WHERE id = 7
  UNION ALL
  SELECT e.id, e.name, s.depth + 1 FROM employees e JOIN sub s ON e.manager_id = s.id)
SELECT * FROM sub;
```

Q55. Number the rows without a window function.

```
SELECT e.id, (SELECT count(*) FROM employees x WHERE x.id <= e.id) AS rn FROM employees e;
```

It is $O(n^2)$, so an interviewer wants to hear why `ROW_NUMBER` is better.

Q56. Cumulative share: which customers make up 80% of revenue (Pareto)?

```
WITH r AS (SELECT customer_id, sum(amount) AS rev FROM orders GROUP BY 1),
     s AS (SELECT *, SUM(rev) OVER (ORDER BY rev DESC ROWS UNBOUNDED PRECEDING) / SUM(rev) OVER
```

```
() AS share FROM r)
SELECT * FROM s WHERE share <= 0.8;
```

Q57. Overlapping date ranges.

```
SELECT a.id, b.id FROM bookings a JOIN bookings b
  ON a.room_id = b.room_id AND a.id < b.id
 AND a.starts_at < b.ends_at AND b.starts_at < a.ends_at;
```

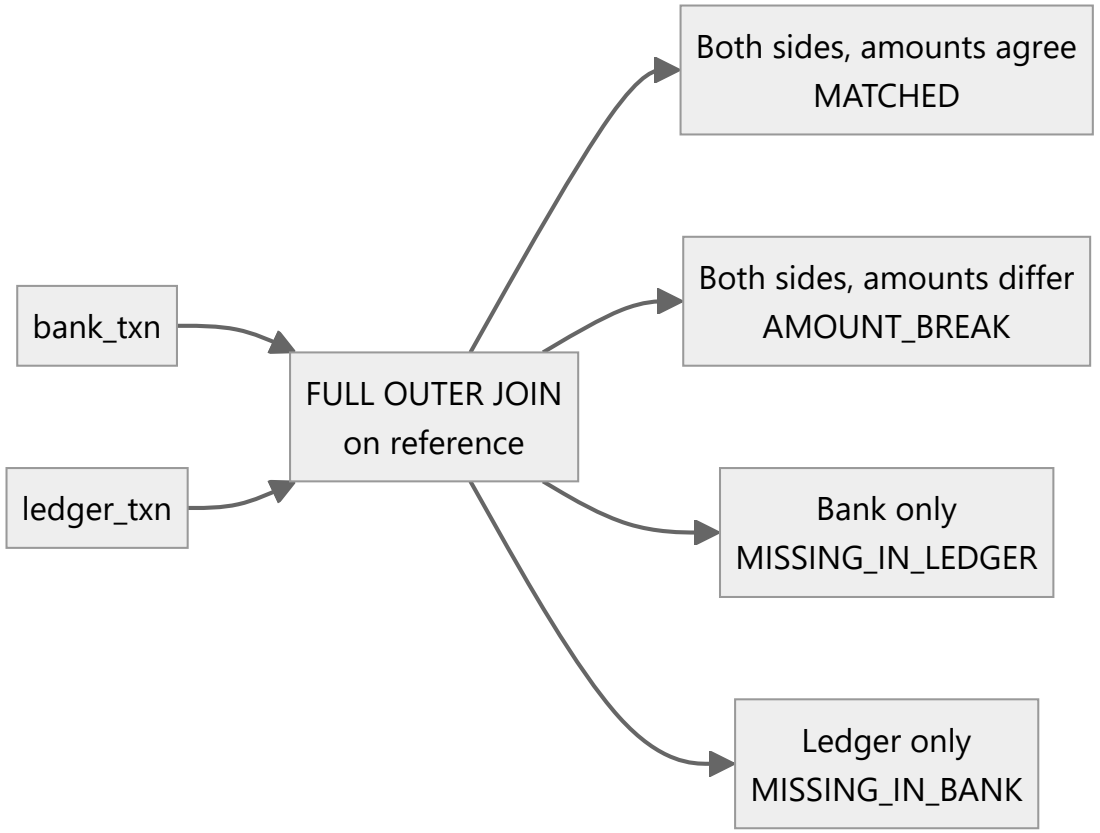
Two ranges overlap when each starts before the other ends. `a.id < b.id` removes self-pairs and mirror duplicates. To forbid overlaps outright, use an `EXCLUDE USING gist` constraint.

Q58. Concurrent sessions at peak.

```
SELECT max(c) FROM (
  SELECT SUM(delta) OVER (ORDER BY t, delta ROWS UNBOUNDED PRECEDING) AS c FROM (
    SELECT started_at AS t, 1 AS delta FROM sessions
    UNION ALL SELECT ended_at, -1 FROM sessions) e) x;
```

Sorting `-1` before `+1` at equal times treats a session ending as the next begins as non-overlapping.

23.15 Reconciliation-flavoured queries



> **RefinePoint:** reconciliation is the product, so these are the queries interviewers should hear you discuss with confidence. Our engine runs matching on DuckDB in-process for large files, with `SqlCompiler` compiling user rules to DuckDB SQL, and matching runs in multiple passes with tolerances. The queries below show the ideas in plain PostgreSQL, using `bank_txn(id, org_id, value_date, reference, amount)` and `ledger_txn(id, org_id, posting_date, reference, amount)`.

Q59. Classify every transaction between bank and ledger.

```sql
SELECT COALESCE(b.reference, l.reference) AS reference, b.amount AS bank, l.amount AS ledger,
       CASE WHEN b.id IS NULL THEN 'MISSING_IN_BANK'
            WHEN l.id IS NULL THEN 'MISSING_IN_LEDGER'
            WHEN b.amount = l.amount THEN 'MATCHED'
            ELSE 'AMOUNT_BREAK' END AS status
FROM (SELECT * FROM bank_txn WHERE org_id = :org) b
FULL OUTER JOIN (SELECT * FROM ledger_txn WHERE org_id = :org) l ON b.reference = l.reference;
```

Filter in subqueries before the join: an `org_id` filter in `WHERE` on a full join would remove the unmatched rows from one side. Also, duplicate references multiply rows, so check for those first.

Q60. Match with an amount tolerance.

```sql
SELECT b.id, l.id, b.amount - l.amount AS diff
FROM bank_txn b JOIN ledger_txn l
  ON b.reference = l.reference AND abs(b.amount - l.amount) <= 0.01;
```

Inclusive, and a tolerance is usually per currency, so join a tolerance table in production. `abs(a - b)` is non-SARGable; the equality on `reference` must do the index work.

Q61. Why does a FULL OUTER JOIN on only a tolerance fail in PostgreSQL?

```sql
-- ERROR: FULL JOIN is only supported with merge-joinable or hash-joinable join conditions
SELECT * FROM bank_txn b FULL JOIN ledger_txn l ON abs(b.amount - l.amount) <= 0.01;
```

PostgreSQL cannot do a full join with no equality condition. Rewrite as a left join plus an anti join:

```sql
SELECT b.id, l.id FROM bank_txn b LEFT JOIN ledger_txn l ON abs(b.amount - l.amount) <= 0.01
UNION ALL
SELECT NULL, l.id FROM ledger_txn l
WHERE NOT EXISTS (SELECT 1 FROM bank_txn b WHERE abs(b.amount - l.amount) <= 0.01);
```

Q62. One-to-one matching with tolerance and date window: each row used once.

```sql
WITH candidates AS (
  SELECT b.id AS bank_id, l.id AS ledger_id, abs(b.amount - l.amount) AS diff,
         abs(b.value_date - l.posting_date) AS days
  FROM bank_txn b JOIN ledger_txn l
    ON abs(b.amount - l.amount) <= 0.01 AND abs(b.value_date - l.posting_date) <= 3),
ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY bank_id   ORDER BY diff, days, ledger_id) AS rb,
```

```
            ROW_NUMBER() OVER (PARTITION BY ledger_id ORDER BY diff, days, bank_id)   AS rl
  FROM candidates)
SELECT bank_id, ledger_id FROM ranked WHERE rb = 1 AND rl = 1;
```

Each side's best choice is kept only when mutual, so nothing pairs twice; rows left over go to the next pass. It is greedy, not globally optimal. A later pass catches rows whose first choice was taken.

Q63. One bank line against many ledger lines (one-to-many by sum).

```
SELECT b.id, array_agg(l.id) AS ledger_ids
FROM bank_txn b JOIN ledger_txn l ON l.reference = b.reference
GROUP BY b.id, b.amount
HAVING abs(b.amount - sum(l.amount)) <= 0.01;
```

Q64. Aging buckets for open breaks.

```
SELECT CASE WHEN age <= 7  THEN '0-7'
            WHEN age <= 30 THEN '8-30'
            WHEN age <= 90 THEN '31-90'
            ELSE '90+' END AS bucket,
       count(*) AS breaks, sum(abs(amount)) AS exposure
FROM (SELECT amount, CURRENT_DATE - opened_on AS age FROM breaks
      WHERE org_id = :org AND status = 'OPEN') b
GROUP BY 1 ORDER BY min(age);
```

`ORDER BY min(age)` keeps the buckets in the right order, which text sorting would not (`'31-90'` sorts before `'8-30'`).

Q65. Duplicates within one side, such as a bank file loaded twice.

```
SELECT reference, value_date, amount, count(*), array_agg(id) AS ids
FROM bank_txn WHERE org_id = :org GROUP BY reference, value_date, amount HAVING count(*) > 1;
```

Q66. Daily control totals: do the two sides net to the same amount?

```
SELECT COALESCE(b.day, l.day) AS day, COALESCE(b.total, 0) AS bank, COALESCE(l.total, 0) AS
ledger,
       COALESCE(b.total, 0) - COALESCE(l.total, 0) AS diff
FROM (SELECT value_date AS day, sum(amount) AS total FROM bank_txn GROUP BY 1) b
FULL JOIN (SELECT posting_date AS day, sum(amount) AS total FROM ledger_txn GROUP BY 1) l ON
b.day = l.day
WHERE COALESCE(b.total, 0) <> COALESCE(l.total, 0);
```

Aggregate *before* joining; otherwise a join on the raw rows multiplies totals (fan-out). Here the `WHERE` is safe because it uses `COALESCE`d values on both sides.

Q67. Match rate per run and trend against the previous run.

```
SELECT run_id, matched, total, round(100.0 * matched / NULLIF(total, 0), 1) AS rate,
       round(100.0 * matched / NULLIF(total, 0), 1)
```

```
       - LAG(round(100.0 * matched / NULLIF(total, 0), 1)) OVER (ORDER BY finished_at) AS
change
FROM run_results WHERE process_id = :p ORDER BY finished_at;
```

Q68. Latest status of each break from a history table (slowly changing data).

```
SELECT DISTINCT ON (break_id) break_id, status, changed_at, changed_by
FROM break_history ORDER BY break_id, changed_at DESC;
```

An index on `(break_id, changed_at DESC)` makes this fast.

Q69. Time from opened to resolved, by month and percentile.

```
SELECT date_trunc('month', resolved_at) AS month,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY resolved_at - opened_at) AS median_time,
       percentile_cont(0.9) WITHIN GROUP (ORDER BY resolved_at - opened_at) AS p90_time
FROM breaks WHERE resolved_at IS NOT NULL GROUP BY 1 ORDER BY 1;
```

Q70. Claim the next queued item safely with SQL alone.

```
UPDATE recon_runs SET status = 'RUNNING', started_at = now()
WHERE id = (SELECT id FROM recon_runs WHERE status = 'QUEUED'
            ORDER BY queued_at LIMIT 1 FOR UPDATE SKIP LOCKED)
RETURNING id;
```

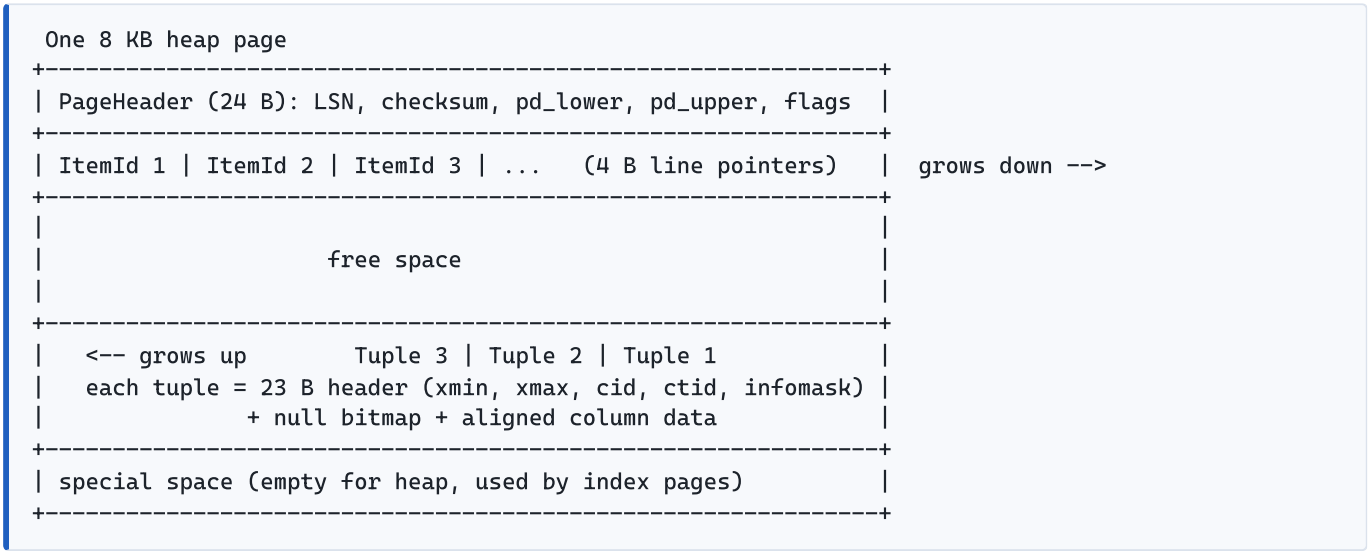
`SKIP LOCKED` lets concurrent workers each take a different row without waiting, and it is what our run and job queues build on (chapter 21). Without it, workers queue on the same row, and in READ COMMITTED a waiter may find the row no longer qualifies and update nothing.

24. Database Internals & Performance (PostgreSQL focus)

Senior interviews treat the database as the place where most production incidents start: a missing index, a bloated table, a connection pool that is too big, a migration that locked a hot table for nine minutes. Interviewers want to hear that you know *why* the database behaves the way it does — how a row is stored, how an index is walked, why an old transaction makes a table grow — because that is what lets you fix problems instead of guessing. This chapter uses PostgreSQL as the reference engine and calls out MySQL/InnoDB differences where interviewers usually probe.

24.1 How PostgreSQL stores a table

A table (a *relation*) is a set of files on disk, each split into **8 KB pages** (blocks). The table's main file is called the **heap**: rows are stored in no particular order, wherever there is free space. Every row version is a **tuple**.



Key facts:

- A row is addressed by its **TID / ctid** = (page number, line-pointer number), e.g. `(42,3)`. Indexes store TIDs, not primary keys.
- Line pointers give one level of indirection: a tuple can move inside the page (compaction) without changing its TID.
- The tuple header holds MVCC information: `xmin` (transaction that created it), `xmax` (transaction that deleted/updated it), `ctid` (pointer to the newer version if updated), and hint bits.
- Column order matters for padding: `bigint, int, bool` packs tighter than `bool, bigint, bool, int` because of 8-byte alignment.

TOAST: what happens to big values

A tuple must fit in a page, but a `text` or `jsonb` value can be up to 1 GB. **TOAST** (The Oversized-Attribute Storage Technique) kicks in when a row exceeds about **2 KB**: Postgres first tries to compress large variable-

length values (pglz, or lz4 from PG14), and if the row is still too big, moves them **out of line** into a hidden `pg_toast.pg_toast_<oid>` table in ~2 KB chunks. The main tuple keeps an 18-byte pointer.

Strategy	Compress?	Move out of line?	Default for
PLAIN	No	No	fixed-length types (`int` , `uuid`)
MAIN	Yes	Only as last resort	`numeric`
EXTERNAL	No	Yes	useful for `text` you `substr()` a lot
EXTENDED	Yes	Yes	`text` , `jsonb` , `bytea`

Consequences: `SELECT *` on a table with large `jsonb` columns pays de-TOAST cost for every row; selecting only the narrow columns is dramatically faster. An update that does not touch the TOASTed column reuses the existing TOAST pointer.

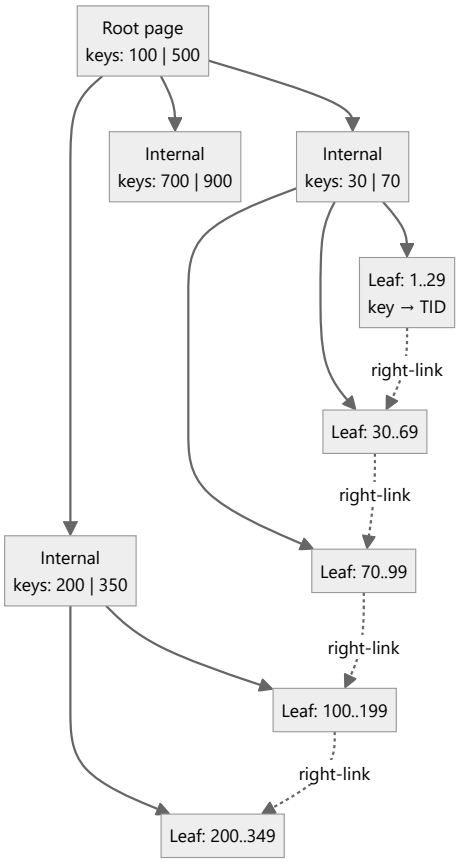
HOT updates and fillfactor

Postgres never updates in place: an `UPDATE` writes a **new tuple version** and marks the old one with `xmax` . Normally every index on the table then needs a new entry pointing to the new TID. A **HOT (Heap-Only Tuple) update** avoids this when (1) no indexed column changed and (2) the new version fits on the **same page**. The old tuple's line pointer then chains to the new one, and indexes keep pointing at the old line pointer. Lowering `fillfactor` (e.g. `ALTER TABLE recon_runs SET (fillfactor = 80)`) leaves 20% free space per page so frequently-updated rows (status, heartbeat) get HOT updates. `pg_stat_user_tables.n_tup_hot_upd` tells you the ratio.

> **RefinePoint:** `recon_runs` rows are updated constantly while a run is active (`heartbeat_at` every 15 seconds, status transitions). None of those columns should be in a non-partial index if we want HOT updates; `heartbeat_at` is not indexed, so the heartbeat updates stay HOT, and the partial indexes are keyed on `status` , which changes only a few times per run.

24.2 B+tree indexes — the default and why

A Postgres B-tree (`nbtree` , a Lehman–Yao **B-link tree**) is a **B+tree**: all keys with their TIDs live in the **leaf** level, internal pages hold only separator keys and child pointers, and leaves are linked left↔right so range scans walk sideways without going back up.

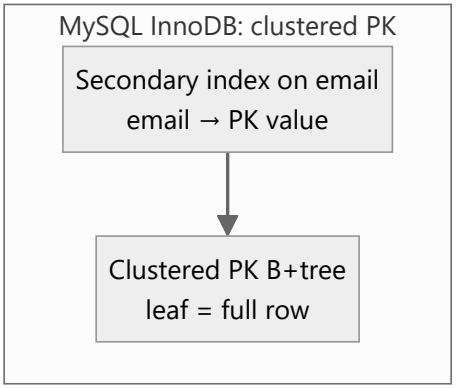


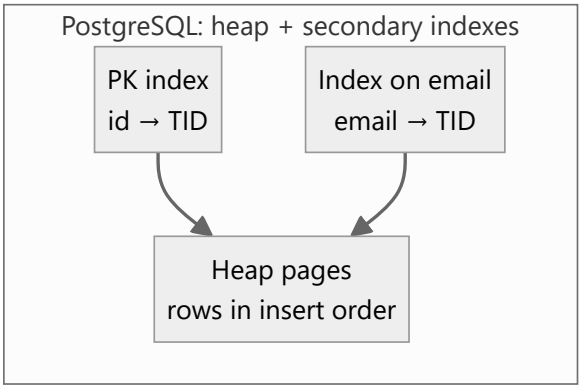
Why B+tree and not a binary tree or hash?

1. **High fan-out:** an 8 KB page holds hundreds of keys, so a tree over 1 billion rows is only 3–4 levels deep. Each level is one page read, and the top levels stay in cache. A binary tree would be ~30 levels.
2. **Disk-friendly:** nodes are page-sized, so one I/O fetches a whole node.
3. **Ordered:** supports `=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `MIN/MAX`, prefix `LIKE 'abc%'` (with `text_pattern_ops` or C collation). A hash index only supports `=`.
4. **Range scans are sequential** along the leaf chain.
5. **Self-balancing** on insert (page splits) so lookups stay O(log n).

Postgres 13+ also **deduplicates** repeated keys in leaves (one key, a posting list of TIDs), which shrinks indexes on low-cardinality columns like `status` or `org_id` considerably.

Clustered vs non-clustered: Postgres heap vs InnoDB





	PostgreSQL	MySQL InnoDB
Table storage	Heap, unordered	The table **is** the PK B+tree (clustered)
Secondary index leaf holds	TID (physical address)	Primary key value
PK lookup	Index → heap (2 hops)	One tree walk
Secondary lookup	Index → heap	Secondary → PK tree (double lookup)
Update moves a row	New TID; all indexes updated unless HOT	PK unchanged; secondary indexes untouched unless their column changed
Random UUID PK	Fine for heap; index pages still split randomly	Painful: random inserts split clustered pages everywhere
`CLUSTER`	One-time physical reorder, not maintained	Always ordered by PK

That is why the InnoDB advice is "keep the PK small and monotonic" (wide PKs are copied into every secondary index), while in Postgres the PK is just another index. Random UUIDv4 keys still hurt B-tree locality in both engines; **UUIDv7** (time-ordered) gives the uniqueness of UUIDs with the insert locality of a sequence.

24.3 Composite, covering, partial and expression indexes

Composite index and the leftmost-prefix rule

An index on `(org_id, process_id, started_at DESC)` is sorted by `org_id`, then `process_id` within each org, then `started_at` within each process — like a phone book sorted by last name, then first name.

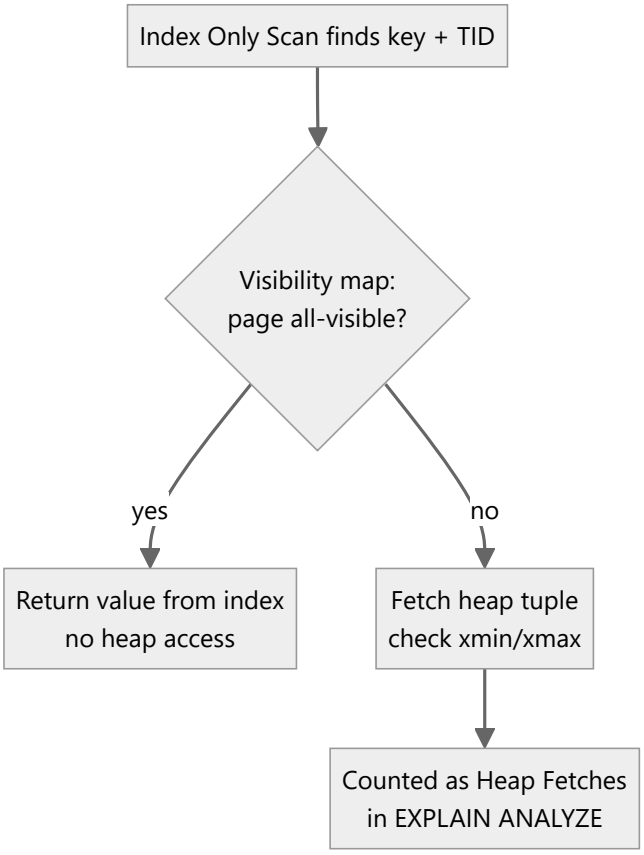
Query predicate	Can use the index?
`org_id = ?`	Yes (leftmost prefix)
`org_id = ? AND process_id = ?`	Yes
`org_id = ? AND process_id = ? ORDER BY started_at DESC LIMIT 20`	Yes — and no sort needed
`process_id = ?`	Not efficiently — a full index scan at best (PostgreSQL 18 adds B-tree skip scan for a low-cardinality leading column)
`org_id = ? AND started_at > ?`	Uses `org_id` to narrow, then filters `started_at` inside the range

Rules of thumb for column order: **equality columns first, then the range or sort column**; among equality columns put the one every query has (the tenant id) first.

> **RefinePoint:** the run-service baseline migration has `CREATE INDEX idx_recon_runs_org_process ON recon_runs (org_id, process_id, started_at DESC)`. The run history screen asks "runs of this process in this workspace, newest first, page of 20" — the index answers it with a single descending range scan and a `LIMIT`, no sort node.

Covering indexes and index-only scans

If every column a query needs is in the index, Postgres can do an **Index Only Scan** and skip the heap. But the index has no visibility information (no `xmin/xmax`), so Postgres must know the row is visible to every transaction. It checks the **visibility map** (2 bits per heap page: *all-visible*, *all-frozen*), maintained by VACUUM.



`INCLUDE` (PG11+) adds non-key payload columns to the leaf only, so they do not affect ordering or uniqueness:

```
CREATE UNIQUE INDEX ux_users_email ON users (email) INCLUDE (display_name, status);
-- SELECT display_name, status FROM users WHERE email = ?  → Index Only Scan
```

A high `Heap Fetches` count in an index-only scan means the table is not vacuumed often enough.

Partial and expression indexes

A **partial index** indexes only rows matching a `WHERE`. It is smaller, cheaper to maintain, and perfect for queues and "active" subsets.

```
-- only queued runs are indexed: the index stays tiny however many runs exist
CREATE INDEX idx_recon_runs_queued ON recon_runs (queued_at) WHERE status = 'QUEUED';
-- unique only among non-deleted rows
CREATE UNIQUE INDEX ux_customer_name ON customers (org_id, lower(name)) WHERE deleted_at IS NULL;
```

The query must contain a predicate the planner can prove implies the index predicate (`WHERE status = 'QUEUED'` literally, not a bind parameter for `status`).

An **expression index** indexes the result of a function: `CREATE INDEX ON users (lower(email))` serves `WHERE lower(email) = ?`. The function must be `IMMUTABLE`.

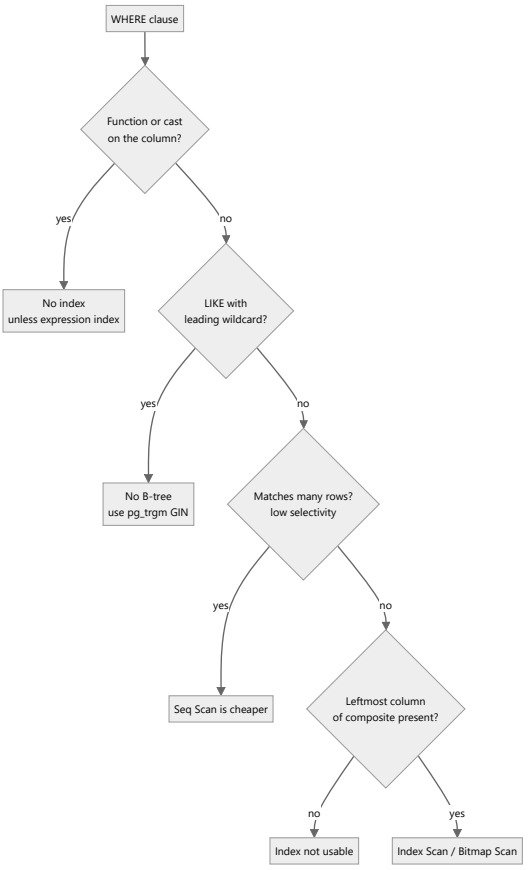
> **RefinePoint:** the run queue (`idx_recon_runs_queued`), the export queue (`idx_run_exports_queue ... WHERE status = 'QUEUED'`) and the "running per workspace" count (`idx_recon_runs_running ON recon_runs (org_id) WHERE status = 'RUNNING'`) are all partial indexes. Finished runs never enter them, so the claim query does not slow down as run history grows.

24.4 Other index types

Type	Structure	Good for	Example
B-tree	Balanced B+tree	Equality, ranges, sorting — the default	`WHERE created_at > ?`
Hash	Hash buckets (WAL-logged since PG10)	Equality only on long values	`WHERE token_hash = ?`
GIN	Inverted index: element → posting list of TIDs	`jsonb @>`, arrays `&&`, full-text `@@`, `pg_trgm` for `LIKE '%x%'`	`WHERE payload @> '{"type":"BANK"}'`
GiST	Generalised balanced tree of bounding predicates	Ranges, geometry, nearest-neighbour, exclusion constraints	no overlapping bookings: `EXCLUDE USING gist (room WITH =, during WITH &&)`
SP-GiST	Space-partitioned (quad-tree, radix)	IP addresses, points, prefixes	`inet` lookups
BRIN	Min/max summary per block range (128 pages)	Huge append-only tables where value correlates with physical order	`audit_events (created_at)` — a few KB for billions of rows

GIN is slow to update (it uses a *pending list* flushed by VACUUM, `fastupdate`) but fast to query; GiST is faster to update, lossy, and needs rechecks. BRIN is useless if rows are not physically ordered by the column.

24.5 When an index is NOT used



1. **Function on the column:** `WHERE date(created_at) = '2026-10-01'` or `lower(email) = ?` cannot use a plain index on the column. Rewrite as a range (`created_at >= '2026-10-01' AND created_at < '2026-10-02'`) or add an expression index.
2. **Implicit cast on the column side:** `WHERE bigint_id = 1.5` or a `numeric` bind parameter makes Postgres cast the column to `numeric` ; in MySQL `WHERE varchar_phone = 12345` casts every row's string to a number. Match the parameter type to the column type (watch JPA/JDBC parameter types).
3. **Leading wildcard:** `LIKE '%bank%'` cannot use B-tree order. Use a `pg_trgm` GIN index or full-text search.
4. **Low selectivity:** if a predicate returns, say, 30% of rows, random heap reads cost more than reading the table sequentially. The planner prices random page reads at `random_page_cost = 4` vs `seq_page_cost = 1` — on SSDs set `random_page_cost` to ~1.1.
5. `OR` **across different columns:** `WHERE a = 1 OR b = 2` needs an index on each column so Postgres can combine them with a **BitmapOr**; otherwise it falls back to a seq scan. Rewriting as `UNION` sometimes helps.
6. `NOT` , `<>` , `IS NOT NULL` **on mostly-non-null columns** — usually low selectivity.
7. **Stale statistics:** the planner thinks a table has 10 rows (just bulk-loaded) and seq-scans; run `ANALYZE` .
8. **Collation:** B-tree on `text` in a non-C collation does not serve `LIKE 'abc%'` ; use `text_pattern_ops` .
9. **Small tables:** a 3-page table is always seq-scanned; that is correct.

24.6 Reading EXPLAIN and EXPLAIN ANALYZE

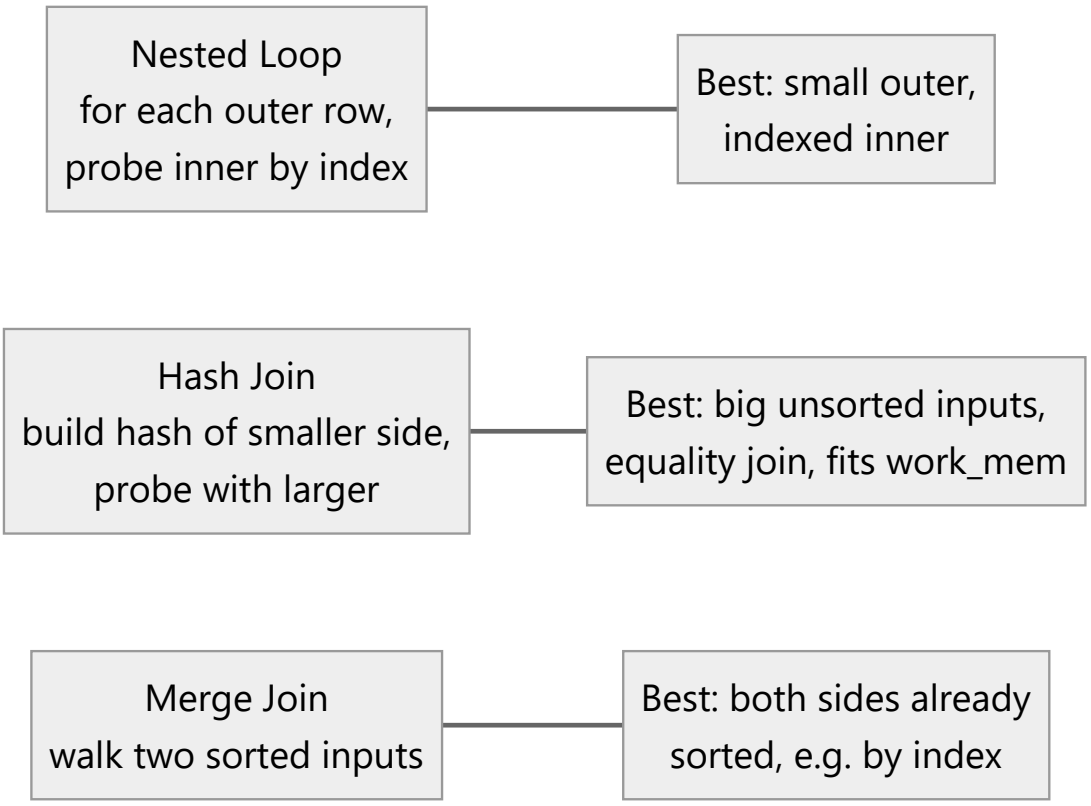
`EXPLAIN` shows the **estimated** plan. `EXPLAIN (ANALYZE, BUFFERS)` **executes** the query and adds actual time, rows and buffer counts (wrap DML in `BEGIN … ROLLBACK`).

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT r.id, r.status, p.name
FROM recon_runs r JOIN recon_processes p ON p.id = r.process_id
WHERE r.org_id = '…' AND r.started_at > now() - interval '7 days';
```

```
Hash Join  (cost=12.4..980.1 rows=310 width=48) (actual time=0.31..4.92 rows=287 loops=1)
  Hash Cond: (r.process_id = p.id)
  Buffers: shared hit=412
  ->  Bitmap Heap Scan on recon_runs r  (cost=8.9..962.0 rows=310 ...) (actual rows=287 loops=1)
        Recheck Cond: ((org_id = '…') AND (started_at > ...))
        ->  Bitmap Index Scan on idx_recon_runs_org_process  (actual rows=287 loops=1)
  ->  Hash  (cost=2.5..2.5 rows=40 ...) (actual rows=40 loops=1)
        ->  Seq Scan on recon_processes p  (actual rows=40 loops=1)
Planning Time: 0.4 ms
Execution Time: 5.1 ms
```

How to read it: plans are trees, read **inside-out / bottom-up**. `cost=startup..total` is in abstract units. Compare **estimated** `rows` **vs actual** `rows` — a 100× mismatch is the root of most bad plans. Multiply actual time by `loops`. `Buffers: shared hit` = from cache, `read` = from disk/OS cache.

Scan node	What it does	When chosen
Seq Scan	Reads every page	Small table or large fraction of rows
Index Scan	Walks index, fetches each heap tuple in index order	Few rows, or ordered output needed
Index Only Scan	Index only, heap only for not-all-visible pages	All needed columns in index
Bitmap Index + Bitmap Heap Scan	Builds a bitmap of matching pages, reads heap in physical order	Medium selectivity, or combining indexes (BitmapAnd/Or)



A hash join that exceeds `work_mem` spills to disk in **batches** (`Batches: 8` in the plan); a sort that spills shows `Sort Method: external merge Disk: 120MB` . Raising `work_mem` for that session/query often fixes it — but `work_mem` is per sort/hash node per connection, so do not raise it globally on a server with 300 connections.

24.7 Statistics and ANALYZE

The planner estimates row counts from statistics collected by `ANALYZE` (run by autovacuum too) into `pg_statistic` , readable via `pg_stats` :

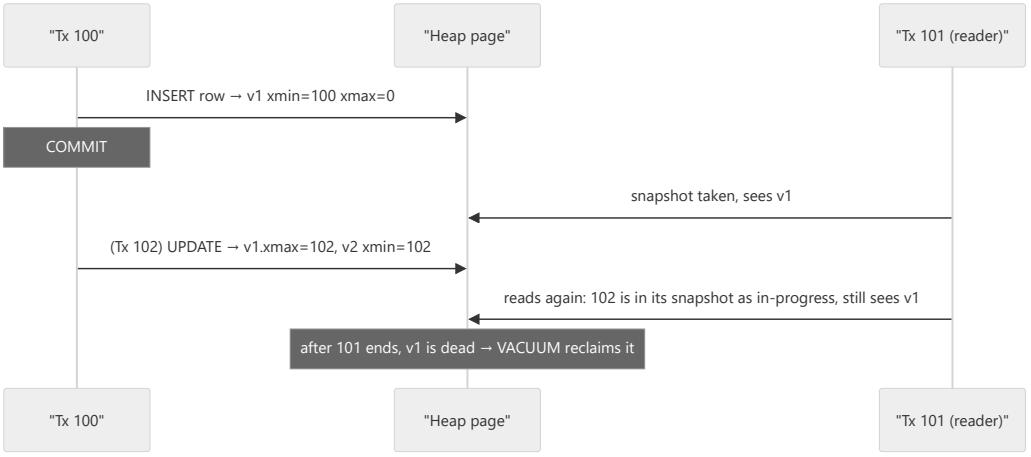
- `null_frac` , `n_distinct` , `avg_width`
- `most_common_vals` + `most_common_freqs` (MCV list)
- `histogram_bounds` (equal-frequency buckets for the rest)
- `correlation` (physical vs logical order: near ±1 makes index range scans cheap)

`ANALYZE` samples `300 × default_statistics_target` rows (100 → 30,000 rows). For skewed columns, raise the target per column: `ALTER TABLE ... ALTER COLUMN org_id SET STATISTICS 1000` . Postgres assumes columns are **independent**: `WHERE city='Chennai' AND state='TN'` is underestimated because it multiplies selectivities. Fix with **extended statistics**: `CREATE STATISTICS s1 (dependencies, ndistinct, mcv) ON city, state FROM addresses` .

> **Trap:** after a bulk load or a big `DELETE` , the stats are wrong until autovacuum's analyze runs. Batch jobs that load then immediately query should run `ANALYZE table` themselves.

24.8 MVCC internals

MVCC (Multi-Version Concurrency Control): readers never block writers and writers never block readers, because every transaction sees a **snapshot** and old row versions are kept until nobody can see them.



- Each tuple has `xmin` (creator xid) and `xmax` (deleter/updater xid, or 0).
- A **snapshot** is `(xmin, xmax, list of in-progress xids)`. A tuple is visible if its `xmin` committed before the snapshot and its `xmax` is empty, aborted, or not yet committed as of the snapshot.
- Commit status lives in the **commit log** (`pg_xact`); the first reader that checks a tuple sets **hint bits** on it (which is why a plain `SELECT` can dirty pages right after a bulk load).
- **Read Committed** takes a new snapshot per *statement*; **Repeatable Read** and **Serializable** take one per *transaction*.
- `UPDATE` = insert new version + set `xmax` on old; `DELETE` = set `xmax`. The old versions are **dead tuples** once no snapshot can see them.

VACUUM, autovacuum and bloat

`VACUUM` (plain) removes dead tuples, makes their space reusable (Free Space Map), updates the **visibility map**, and **freezes** old tuples. It takes only a `SHARE UPDATE EXCLUSIVE` lock — reads and writes continue. It rarely shrinks the file; space is reused, not returned. `VACUUM FULL` rewrites the table into a new file and takes `ACCESS EXCLUSIVE` (blocks everything); prefer `pg_repack` online.

Autovacuum triggers per table when dead tuples exceed `autovacuum_vacuum_threshold (50) + autovacuum_vacuum_scale_factor (0.2) × reltuples`. On a 100-million-row table that is 20 million dead rows before cleanup — far too late. Tune per table:

```
ALTER TABLE outbox_events SET (autovacuum_vacuum_scale_factor = 0.01,
autovacuum_vacuum_cost_limit = 2000);
```

Bloat = space held by dead tuples or empty pages. Causes: autovacuum too slow, and above all **anything holding back the xmin horizon**: long-running transactions, `idle in transaction` sessions, abandoned replication slots, `hot_standby_feedback` with long replica queries, forgotten prepared transactions. VACUUM cannot remove a dead tuple that some open snapshot might still need.

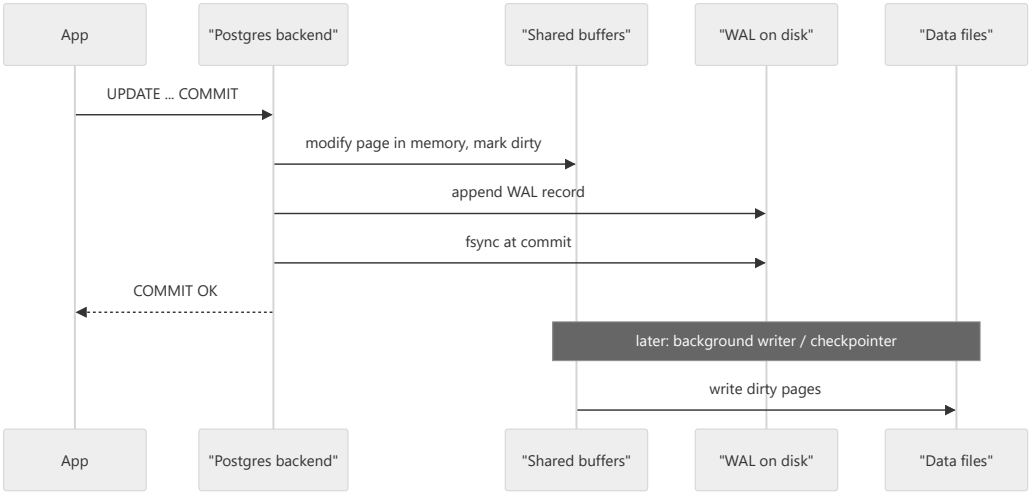
> **RefinePoint:** the outbox and queue tables are classic "high churn" tables — rows are inserted, updated to sent/running, then deleted by `OutboxRelay.purge()` (sent events after 7 days, consumed-event ids after 30). Without aggressive per-table autovacuum settings, the queue index would fill with dead entries and `SKIP LOCKED` claims would slow down scanning past them.

Transaction ID wraparound

XIDs are **32-bit**; Postgres compares them modulo 2^{32}, so each transaction sees ~2 billion xids "in the past" and ~2 billion "in the future". A tuple that is never **frozen** would one day appear to be in the future and vanish. VACUUM therefore **freezes** old tuples (marks them as visible to everyone). Autovacuum forces an anti-wraparound vacuum when a table's `relfrozenxid` age passes `autovacuum_freeze_max_age` (200 million). If it still cannot keep up, Postgres eventually refuses new transactions ("database is not accepting commands to avoid wraparound data loss") until a vacuum finishes. Monitor `age(datfrozenxid)` per database.

24.9 WAL, durability and checkpoints

Write-Ahead Logging: every change is first written to the **WAL** (sequential log in `pg_wal`); data pages are modified in shared buffers and written to disk later. On `COMMIT`, only the WAL needs to be flushed (`fsync`) — that is the durability point.



- **Checkpoints** (every `checkpoint_timeout` = 5 min or when `max_wal_size` is reached) flush all dirty pages and record a redo point. Crash recovery replays WAL from the last checkpoint.
- **Full-page writes:** the first change to a page after a checkpoint writes the whole page into WAL, protecting against torn pages. This is why too-frequent checkpoints inflate WAL volume.
- **Group commit:** concurrent commits share one fsync.
- `synchronous_commit = off` returns before the WAL flush — you may lose the last few hundred ms of commits on crash but never corrupt data. Good for logs/telemetry, not for money.
- WAL is also the source for **replication** and **point-in-time recovery** (base backup + archived WAL).

24.10 Locking

Row locks are stored in the tuple header (`xmax` + infomask), not in shared memory, so locking a million rows does not exhaust lock memory.

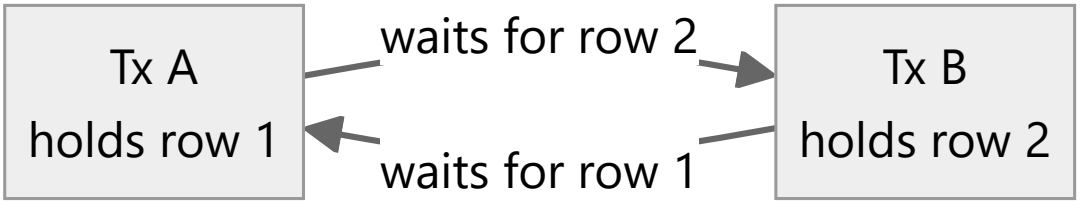
Row lock	Taken by	Conflicts with
`FOR KEY SHARE`	FK checks	`FOR UPDATE`
`FOR SHARE`	explicit	`FOR NO KEY UPDATE` , `FOR UPDATE`
`FOR NO KEY UPDATE`	ordinary `UPDATE` not changing a key	`FOR SHARE` , itself, `FOR UPDATE`
`FOR UPDATE`	`DELETE` , key updates, explicit	all

Table lock modes (most important ones):

Mode	Taken by	Blocks
ACCESS SHARE	`SELECT`	only ACCESS EXCLUSIVE
ROW EXCLUSIVE	`INSERT/UPDATE/DELETE`	SHARE and stronger
SHARE UPDATE EXCLUSIVE	`VACUUM` , `ANALYZE` , `CREATE INDEX CONCURRENTLY`	itself, schema changes
SHARE	`CREATE INDEX` (non-concurrent)	writes
ACCESS EXCLUSIVE	most `ALTER TABLE` , `DROP` , `TRUNCATE` , `VACUUM FULL`	everything, including `SELECT`

The lock queue trap: `ALTER TABLE` waits for a long `SELECT` to finish; while it waits in the queue, every new `SELECT` queues *behind it*. A 1-second DDL becomes a full outage. Always `SET lock_timeout = '3s'` before DDL and retry.

Deadlocks: a waiting backend that has waited `deadlock_timeout` (1 s) builds the wait-for graph; if it finds a cycle, it aborts one transaction with `40P01 deadlock detected` .



Prevention: lock rows in a consistent order (`ORDER BY id FOR UPDATE`), keep transactions short, retry on `40P01` .

Advisory locks are application-defined locks on a 64-bit key: `pg_advisory_lock(key)` (session-level, must be released), `pg_advisory_xact_lock(key)` (released at commit/rollback), and `pg_try_advisory_lock` (non-blocking).

> **RefinePoint:** run-service serialises the claim step with `SELECT pg_advisory_xact_lock(hashtext('run-queue'))` and then `... LIMIT 1 FOR UPDATE OF r SKIP LOCKED`. The advisory lock makes the per-workspace concurrency cap exact across instances (two instances cannot both see "2 running, cap 3" and both claim), and it is held for milliseconds. Run numbers per process use `pg_advisory_xact_lock(hashtext('run-number:' || processId))`, and identity-service uses one keyed by the Keycloak id to make workspace provisioning idempotent under concurrent first logins.

24.11 Isolation levels and how Postgres implements them

Level	Postgres behaviour	Anomalies still possible
Read Uncommitted	Same as Read Committed	—
Read Committed (default)	Snapshot per statement; an `UPDATE` on a row changed by a concurrent committed tx re-evaluates the WHERE on the new version	Non-repeatable read, phantom, lost update (read-modify-write)
Repeatable Read	Snapshot per transaction (Snapshot Isolation); concurrent update of same row → `40001`	Write skew
Serializable	SSI: Snapshot Isolation + dependency tracking	None (aborts with `40001` instead)

SSI (Serializable Snapshot Isolation, PG 9.1+): transactions run on snapshots as in Repeatable Read, but Postgres also records **SIREAD locks** (what each transaction *read*, at tuple/page/relation granularity) — these never block. When a write touches something another concurrent transaction read, it records a **read-write antidependency**. If it finds a "dangerous structure" (two consecutive rw-edges among concurrent transactions, which every non-serializable cycle contains), it aborts one with `could not serialize access`. The application **must retry** serialization failures — this is the price of no blocking.

Write skew example: two doctors both check "at least 2 on call", both see 2, each removes themselves → 0 on call. Repeatable Read allows it; Serializable aborts one.

24.12 Connection pooling

Every Postgres connection is a **process** (~5–10 MB, plus per-query memory). Thousands of connections cause context switching and lock contention; throughput *drops* past a point.

HikariCP sizing: the well-known starting formula is `connections = (core_count × 2) + effective_spindle_count`. For a 4-core DB server with SSD that is ~10 connections **in total across all app instances**, not per instance. Pool sizing is about the database's capacity: a small pool with a queue of waiting threads beats a large pool of thrashing connections. To avoid pool-induced deadlock when a thread needs several connections at once: `pool ≥ threads × (connections_per_thread − 1) + 1`.

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 10          # fixed-size pool (Hikari recommends)
```

```
      connection-timeout: 3000  # fail fast instead of piling up requests
      max-lifetime: 1800000     # below any DB/firewall idle cut-off
      leak-detection-threshold: 20000
```

pgBouncer sits between apps and Postgres and multiplexes many client connections onto few server connections:

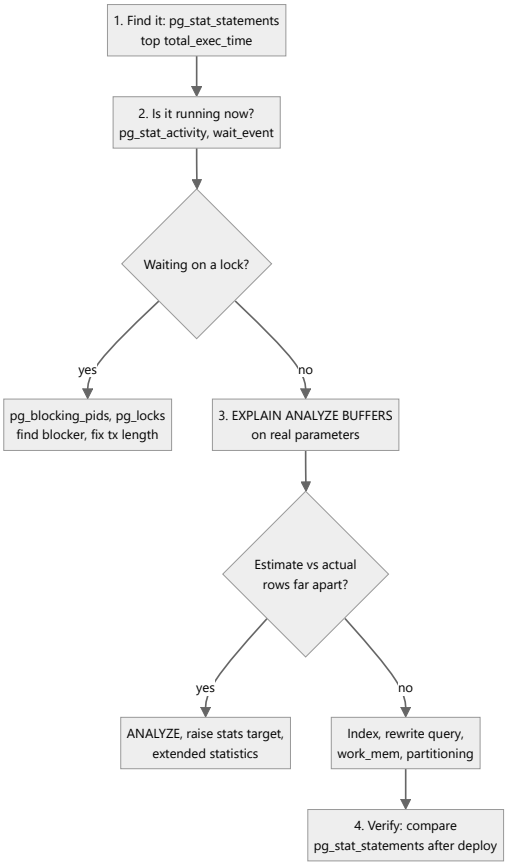
Mode	Server connection held for	Breaks
Session	The client session	nothing, but little multiplexing
Transaction	One transaction	session state: `SET`, session advisory locks, `LISTEN/NOTIFY`, temp tables, server-side prepared statements (unless pgBouncer ≥ 1.21 with `max_prepared_statements`, or JDBC `prepareThreshold=0`)
Statement	One statement	multi-statement transactions

With 13 services × N instances × pool size, the total quickly exceeds `max_connections`; transaction-mode pgBouncer is the usual answer. Note that **transaction-scoped** advisory locks (`pg_advisory_xact_lock`) are safe in transaction mode, session-scoped ones are not.

24.13 N+1 at the database level

N+1: one query loads N parents, then N more queries load each child. Each query is fast, so nothing looks slow in isolation, but 1,001 round trips × 1 ms network = a 1-second page. At the DB level you detect it in `pg_stat_statements` as a query with enormous `calls` and tiny `mean_exec_time`, or in logs as the same statement repeated with different ids. Fixes: join fetch / `@EntityGraph`, `@BatchSize` (turns N into N/size queries with `IN (...)`), a single query with `WHERE parent_id = ANY(?)`, or a projection DTO query. (JPA specifics are in the Hibernate chapter.)

24.14 Slow-query troubleshooting playbook




```
-- top 10 by total time (the queries that cost the system the most)
SELECT queryid, calls, round(total_exec_time) AS total_ms, round(mean_exec_time, 2) AS mean_ms,
       rows, shared_blks_hit, shared_blks_read, left(query, 120)
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

-- who blocks whom right now
SELECT pid, pg_blocking_pids(pid) AS blocked_by, state, wait_event_type, now() - xact_start AS
tx_age, left(query, 80)
FROM pg_stat_activity WHERE datname = current_database() AND state <> 'idle' ORDER BY tx_age
DESC;
```

Also turn on `log_min_duration_statement = 500ms`, `log_lock_waits = on`, and `auto_explain` (logs plans of slow queries with their real parameters — essential because a plan with a generic parameter can differ from a custom one).

24.15 Partitioning

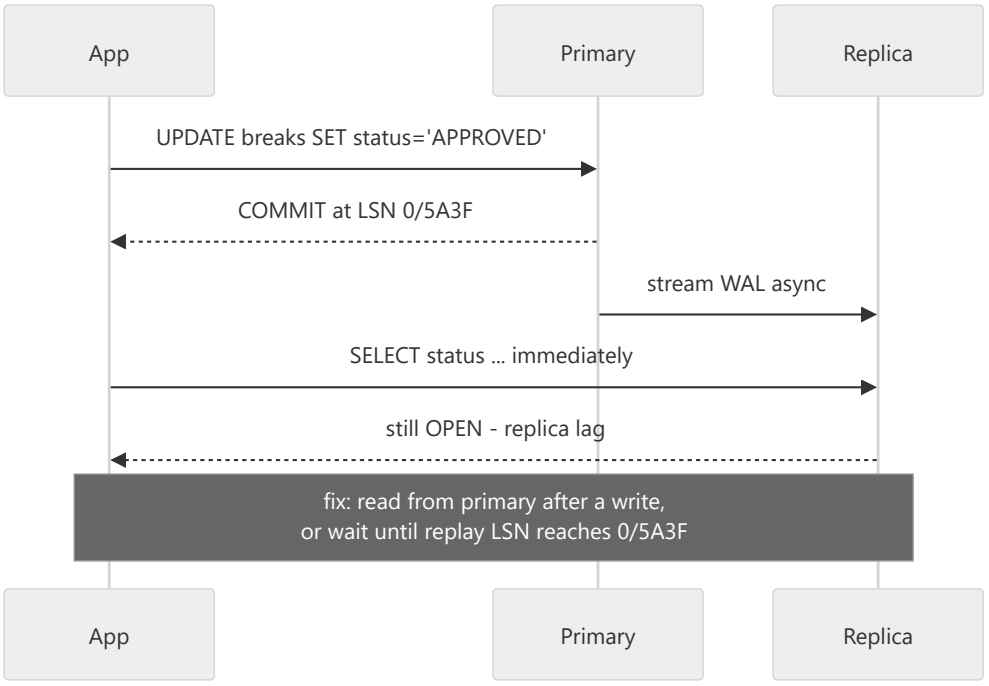
Declarative partitioning (PG10+) splits one logical table into child tables:

Kind	Key	Typical use
RANGE	`created_at`	Time series, audit logs, events — drop old months instantly
LIST	`region`, `tenant tier`	Known discrete values
HASH	`org_id` modulus N	Spread write load evenly when no natural range

```
CREATE TABLE audit_events (id uuid, org_id uuid, created_at timestamptz NOT NULL, payload jsonb,
                           PRIMARY KEY (id, created_at)) PARTITION BY RANGE (created_at);
CREATE TABLE audit_events_2026_11 PARTITION OF audit_events
  FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
ALTER TABLE audit_events DETACH PARTITION audit_events_2025_11 CONCURRENTLY; -- PG14+
```

Benefits: **partition pruning** (planner/executor skips partitions that cannot match), retention by `DROP` / `DETACH` instead of a huge `DELETE` (no bloat), smaller per-partition indexes and vacuums. Costs: unique constraints and PKs **must include the partition key**; queries without the key scan every partition; thousands of partitions slow planning. Partition when a table is big *and* queries/retention align with the key — not by default.

24.16 Replication and read replicas



- **Streaming (physical) replication:** the standby replays the primary's WAL byte for byte; whole-cluster copy, read-only, same major version. **Asynchronous** by default (replica lag, possible loss of last commits on failover); **synchronous** (`synchronous_standby_names`) waits for the standby's flush — no loss, higher commit latency.
- **Logical replication:** publishes row changes per table (publication/subscription); works across major versions, can replicate a subset — used for upgrades and CDC (Debezium reads the logical decoding stream).
- **Replica lag** is measured by `pg_stat_replication` (`replay_lag`) on the primary or `now() - pg_last_xact_replay_timestamp()` on the replica.
- **Read-your-writes** strategies: route the user's reads to the primary for a few seconds after they write (sticky flag in session); or record the commit LSN (`pg_current_wal_lsn()`) and read from a replica only when `pg_last_wal_replay_lsn() >= that LSN` ; or send all reads in a transaction that wrote to the primary (Spring `@Transactional(readOnly = true)` + routing datasource sends only read-only transactions to replicas).
- Long queries on a replica can be cancelled ("canceling statement due to conflict with recovery") when the primary vacuums rows they need; `hot_standby_feedback = on` prevents that but causes bloat on the

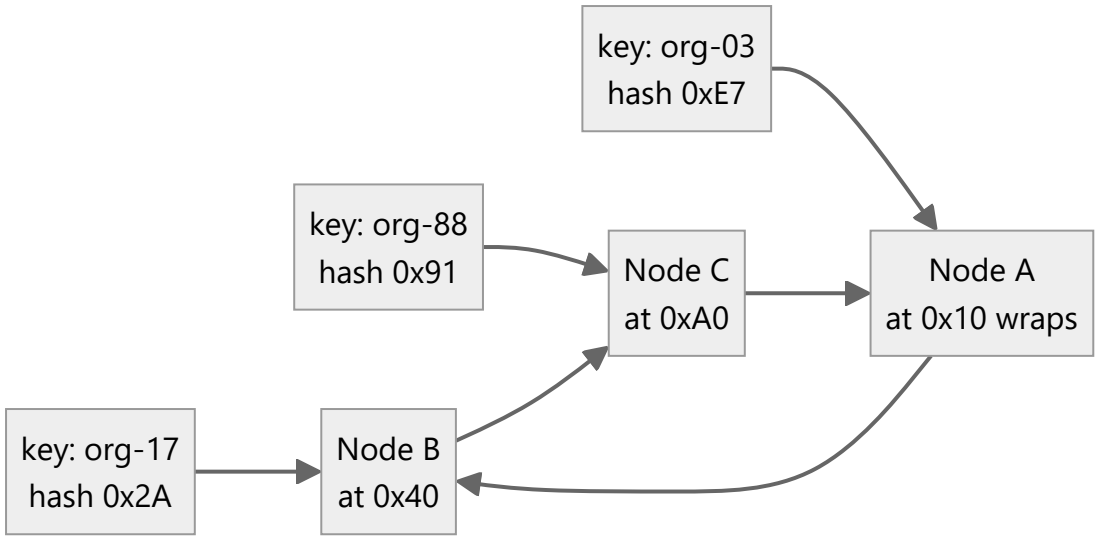
primary.

24.17 Sharding, consistent hashing, CAP and PACELC

Sharding splits data across independent databases by a shard key.

Strategy	How	Pro	Con
Range	`org_id` A–M on shard 1	Range queries, simple	Hotspots, rebalancing
Hash	`hash(org_id) % N`	Even spread	Changing N moves almost all keys
Directory / lookup	Table maps tenant → shard	Move tenants individually, place big tenants alone	Lookup service is critical
Geo	By region	Data residency	Cross-region queries

Consistent hashing puts both nodes and keys on a hash ring; a key belongs to the first node clockwise. Adding a node moves only the keys between it and its predecessor (~1/N of keys). **Virtual nodes** (each physical node at many ring positions) smooth out the distribution.



The right shard key for multi-tenant SaaS is almost always the **tenant id**, so one tenant's queries hit one shard (Citus calls this a distribution column).

CAP: during a network **P**artition, a distributed store must choose **C**onsistency (refuse some requests) or **A**vailability (answer, possibly stale). **PACELC** extends it: *if Partition → A or C; Else (normal operation) → Latency or Consistency.* A single Postgres primary with sync replicas is PC/EC; Cassandra and DynamoDB defaults are PA/EL; MongoDB with majority writes leans PC/EC.

24.18 OLTP vs OLAP: row storage vs columnar storage

```
Row store (Postgres heap): one page holds whole rows
  [id=1 | date | amount | ref | desc] [id=2 | date | amount | ref | desc] ...
  SUM(amount) over 10M rows reads every column of every row.
```

```
Column store (DuckDB, ClickHouse, Parquet): one block holds one column
  id:     [1, 2, 3, 4, ...]
  amount: [10.00, 25.50, 25.50, 99.00, ...]   ← compressed, read only this
  ref:    ["INV-1", "INV-2", ...]
  SUM(amount) reads one column, vectorized over 2048-value chunks.
```

	OLTP (Postgres)	OLAP (DuckDB, warehouses)
Workload	Many small reads/writes by key	Few huge scans, joins, aggregates
Layout	Row-oriented	Column-oriented, compressed (dictionary, RLE)
Execution	Tuple-at-a-time	Vectorised batches, SIMD, parallel
Concurrency	High, MVCC, row locks	Low, bulk
Indexes	Essential	Mostly zone maps / min-max

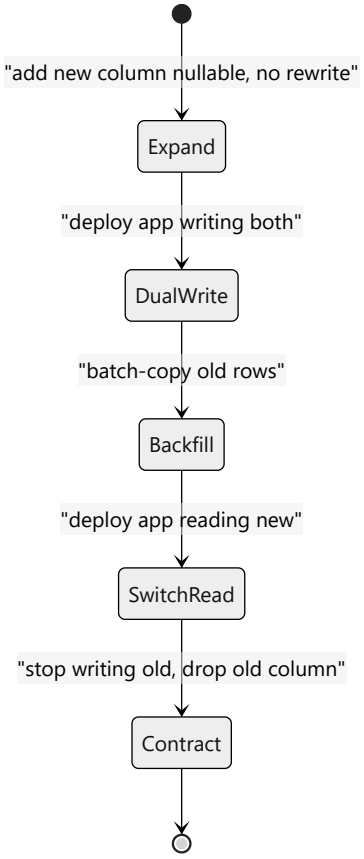
> **RefinePoint:** the platform data (workspaces, processes, runs, breaks, approvals) is classic OLTP in Postgres. Matching a large bank statement against a ledger is OLAP-shaped: big joins on keys, tolerance comparisons, aggregations. `ryfn-engine` loads both sides into **DuckDB in-process** and `ryfn-expression`'s `SqlCompiler` turns the user's match rules into DuckDB SQL. Columnar vectorised joins run in the worker's own process, avoid row-by-row Java and network round trips, and put no load on the shared OLTP database. Only the results (per-run `live_*` tables in the `run` schema) and summaries are written back to Postgres.

24.19 Schema migrations with Flyway, without downtime

Flyway applies versioned scripts (`V1__baseline.sql`, `V2__add_queue.sql`) in order, records each in `flyway_schema_history` with a checksum, and refuses to start if an applied script was edited (`validate`). Repeatable scripts (`R__views.sql`) rerun when their checksum changes.

> **RefinePoint:** each service owns its schema and its Flyway history. Before release 1 we squashed every service's history into a single `V1` baseline; from now on every change is a new `V2+` script — applied scripts are never edited.

Expand/contract (parallel change) lets old and new application versions run side by side during a rolling deploy:



Zero-downtime rules for Postgres:

Change	Safe way
Add column	Nullable or with a constant default (PG11+: metadata only, no rewrite)
Add NOT NULL	Add `CHECK (col IS NOT NULL) NOT VALID`, then `VALIDATE CONSTRAINT` (weak lock), then `SET NOT NULL` (PG12+ uses the validated check, no scan)
Add index	`CREATE INDEX CONCURRENTLY` in its own migration (cannot run inside a transaction; Flyway runs it non-transactionally, do not mix with other statements)
Add foreign key	`ADD CONSTRAINT … NOT VALID`, then `VALIDATE CONSTRAINT`
Rename column/table	Expand/contract: new column, dual write, backfill, switch, drop
Change type	New column + backfill; `ALTER TYPE` rewrites the table under ACCESS EXCLUSIVE
Backfill	Batches of a few thousand rows with commits between, not one giant `UPDATE`
Any DDL	`SET lock_timeout = '3s'` first, retry on timeout

24.20 Multi-tenant indexing

In a shared-schema multi-tenant database every query filters by tenant, so **the tenant column leads every composite index**: `(org_id, status, process_id)`, `(org_id, journal_date DESC)`. Benefits: each tenant's rows form a contiguous index range; queries never touch another tenant's entries; a missing `org_id` predicate shows up in `EXPLAIN` immediately as a scan over all tenants. Unique constraints are per tenant: `UNIQUE (org_id, lower(name))`. Statistics on `org_id` are skewed (one big tenant, many small ones) — raise the

statistics target so the planner knows a big tenant needs a different plan. For defense in depth, Postgres **Row-Level Security** policies (`USING (org_id = current_setting('app.org_id')::uuid)`) can enforce isolation inside the DB.

> **RefinePoint:** every tenant table extends `BaseOrgEntity` (an `org_id` column) and the indexes follow the rule — `idx_open_items_org (org_id, status, process_id)` , `idx_journals_org (org_id, status, journal_date DESC)` , `idx_run_export_requests_user (org_id, user_id, requested_at DESC)` . Tenant checks happen in the application (`OrgAccessInterceptor` , `OrgContextService`), and the index shape guarantees that a correctly-scoped query is also a fast one.

24.21 Interview questions and answers

Q1. How is a row physically stored in PostgreSQL?

As a tuple inside an 8 KB heap page, addressed by a TID `(page, line pointer)` . The page has a header, an array of line pointers growing from the top, tuples growing from the bottom, and free space between. Each tuple has a 23-byte header with `xmin` , `xmax` , `ctid` and flag bits, then a null bitmap and the column values aligned to their type's boundary. Rows are not kept in key order — the heap is unordered.

Q2. What is TOAST and when does it matter for performance?

TOAST compresses and/or moves large variable-length values (over roughly 2 KB per row) into a separate `pg_toast` table in chunks. It matters because reading a TOASTed column costs extra page reads and decompression: `SELECT *` on a table with big `jsonb` payloads can be 10× slower than selecting the narrow columns. It also means wide values do not bloat the main heap, so scans over the other columns stay fast.

Q3. What is a HOT update?

A Heap-Only Tuple update writes the new row version on the same page and links it from the old one, without adding entries to any index. It happens only if no indexed column changed and the page has room. HOT updates cut index bloat and write amplification dramatically; you encourage them with a lower `fillfactor` and by not indexing frequently-updated columns.

Q4. Why do databases use B+trees for indexes instead of binary search trees or hash tables?

Because B+trees have very high fan-out (hundreds of keys per page), so they are only 3–4 levels deep for billions of rows, each level is one page I/O, and the leaves are sorted and linked, giving efficient range scans and ordered output. Binary trees are deep and not page-aligned; hash tables support only equality and cannot serve `ORDER BY` , ranges or prefix searches.

Q5. What is the difference between a B-tree and a B+tree?

In a B-tree, keys and data can live in internal nodes too; in a B+tree, all data (key + row pointer) is in the leaves and internal nodes hold only separators. B+tree internal nodes therefore fit more keys (higher fan-out) and

every lookup ends at a leaf, while linked leaves make range scans sequential. Database "B-tree indexes" are B+trees in practice.

Q6. Clustered vs non-clustered index — how do Postgres and MySQL differ?

In InnoDB the table *is* a clustered B+tree on the primary key: leaves contain whole rows and secondary indexes store the PK value, so a secondary lookup is two tree walks. Postgres has no clustered index: the table is a heap and every index, including the PK, stores TIDs pointing into it. `CLUSTER` in Postgres reorders the heap once but is not maintained. Consequence: InnoDB wants a small, monotonic PK; in Postgres the PK is just another index, but an update that cannot be HOT touches every index.

Q7. Is UUID a bad primary key?

Random UUIDv4 keys spread inserts randomly across the index, causing page splits, poor cache locality and larger indexes (16 bytes vs 8). In InnoDB this is worse because the whole table is clustered on it. Advantages are global uniqueness, no coordination, safe to expose and merge across services. **UUIDv7** keeps the advantages and inserts in time order. Postgres stores `uuid` natively in 16 bytes — never store UUIDs as `varchar(36)`.

Q8. Explain the leftmost-prefix rule.

A composite index `(a, b, c)` is sorted by `a`, then `b`, then `c`, so it can be used for predicates on `a`, `a+b`, or `a+b+c`, and for ordering by those columns in that order, but not efficiently for `b` or `c` alone. A range condition on a column stops the index from narrowing on the columns after it (they become filters inside the range). So put equality columns first and the range/sort column last.

Q9. What is a covering index and an index-only scan?

A covering index contains every column a query needs, so Postgres can answer from the index alone (Index Only Scan). Use `INCLUDE` for payload columns that are not search keys. It only avoids the heap for pages marked all-visible in the visibility map, so tables that are not vacuumed show many `Heap Fetches` and lose the benefit.

Q10. What are partial indexes and where are they useful?

Indexes with a `WHERE` clause that cover only a subset of rows. Ideal for queues (`WHERE status = 'QUEUED'`), soft deletes (`WHERE deleted_at IS NULL`), and per-subset uniqueness. They are small, cheap to maintain and stay small as history grows. The query predicate must logically imply the index predicate, so use a literal, not a bind parameter, for the filtered column.

Q11. When would you use GIN, GiST, BRIN or hash?

GIN for "contains" queries on multi-valued data: `jsonb @>`, arrays, full-text search, trigram `LIKE '%x%'`. GiST for ranges, geometry, nearest-neighbour and exclusion constraints (no overlapping periods). BRIN for very large append-only tables where the column follows physical order, such as timestamps in logs — tiny index, coarse filtering. Hash for pure equality on large keys (slightly smaller than B-tree), rarely needed.

Q12. Why is my index not used? List the usual causes.

Function or cast on the indexed column; type mismatch between parameter and column; leading wildcard `LIKE`; low selectivity so seq scan is cheaper; `OR` across columns without indexes on each; missing leading column of a composite; stale statistics; non-C collation for prefix `LIKE`; tiny table. Diagnose with `EXPLAIN ANALYZE` and check estimated vs actual rows first.

Q13. `WHERE created_at::date = '2026-10-01'` is slow. Fix it.

The cast on the column prevents use of the index on `created_at`. Rewrite as a half-open range: `WHERE created_at >= '2026-10-01' AND created_at < '2026-10-02'`. With `timestamptz`, be explicit about the time zone of the boundaries. Alternatively add an expression index, but the range rewrite is better because the same index also serves other range queries.

Q14. Can an index help `ORDER BY ... LIMIT`?

Yes, and it is one of the biggest wins: if an index matches the filter and the sort order, Postgres walks the index and stops after `LIMIT` rows — no sort of the full result. `ORDER BY started_at DESC LIMIT 20` with index `(org_id, process_id, started_at DESC)` reads ~20 index entries. B-trees can be walked backwards, so `ASC` indexes also serve `DESC`, but mixed directions (`a ASC, b DESC`) need a matching index definition.

Q15. What is the cost of having too many indexes?

Every insert and non-HOT update writes to every index (write amplification, WAL volume), indexes compete for cache, they prevent HOT updates if they include updated columns, and they slow down VACUUM. Find unused ones with `pg_stat_user_indexes.idx_scan = 0` (over a long enough window, and checking replicas too) and drop them.

Q16. How do you read an EXPLAIN ANALYZE output?

Read the tree from the innermost nodes outward. For each node compare estimated rows with actual rows (big gaps mean bad statistics or correlated predicates), multiply actual time by loops, look at the node consuming most time, and check `Buffers` for disk reads. Look for seq scans on big tables with selective filters, `Rows Removed by Filter` in the millions, sorts or hashes spilling to disk, and nested loops with huge outer row counts.

Q17. Seq scan vs index scan vs bitmap scan?

Seq scan reads every page in order — best when a large fraction of rows match. Index scan walks the index and fetches heap tuples one by one in index order — best for few rows or when order is needed. Bitmap scan collects matching TIDs from one or more indexes into a page bitmap, then reads those heap pages in physical order — best for medium selectivity and for combining indexes with AND/OR. Bitmap scans need a `Recheck Cond` when the bitmap becomes lossy (page-level).

Q18. Nested loop vs hash join vs merge join?

Nested loop: for each outer row, look up matches in the inner (ideally via index); best when the outer side is small. Hash join: build an in-memory hash table on the smaller input, stream the larger; best for large, unsorted equality joins, needs `work_mem`. Merge join: walk two inputs sorted on the join key; best when both are already sorted (indexes) or for very large inputs, and supports range-ish join conditions. A nested loop chosen because the planner underestimated the outer side (estimated 1, actual 50,000) is a classic slow plan.

Q19. How does the planner estimate row counts?

From `pg_statistic`: null fraction, number of distinct values, most common values with frequencies, histogram bounds and correlation, collected by `ANALYZE` from a sample. It multiplies selectivities of predicates assuming independence, which underestimates correlated columns. Remedies: `ANALYZE`, higher `STATISTICS` target for skewed columns, and `CREATE STATISTICS` for dependent column groups.

Q20. A query was fast yesterday and is slow today with no code change. What do you check?

Plan change: compare `EXPLAIN` now vs before (`auto_explain` logs help). Typical causes: statistics changed after data growth or a bulk load, a generic prepared-statement plan replacing custom ones after five executions, table/index bloat from blocked vacuum, a parameter value hitting a big tenant, lock waits (check `pg_stat_activity.wait_event`), or a resource issue (I/O saturation, checkpoints, a replica rebuild). Fix the cause rather than adding hints.

Q21. Explain MVCC in PostgreSQL.

Each write creates a new tuple version stamped with the writing transaction's xid (`xmin`), and the old version gets `xmax`. Each transaction reads using a snapshot that defines which xids are committed for it, so readers see a consistent state without locking and writers do not block readers. Old versions become dead once no snapshot needs them and are removed by VACUUM. InnoDB, by contrast, updates in place and keeps old versions in undo logs; readers reconstruct old versions from undo.

Q22. Why does a table grow even though the row count is constant?

Every update and delete leaves a dead tuple; if VACUUM cannot reclaim them as fast as they are created — or something holds back the xmin horizon — the table and its indexes bloat. Check `n_dead_tup` and `last_autovacuum` in `pg_stat_user_tables`, and look for old transactions in `pg_stat_activity` (`xact_start`), `idle in transaction` sessions, inactive replication slots, and prepared transactions.

Q23. How does a long-running transaction hurt the database?

It pins the xmin horizon: no dead tuple newer than its snapshot can be vacuumed anywhere in the database, so hot tables bloat, index scans slow down, and freeze is delayed. It also holds locks that can block DDL (and everything queued behind the DDL). Set `idle_in_transaction_session_timeout`, keep transactions short, and never do HTTP calls or user think-time inside a transaction.

Q24. What is transaction ID wraparound and how do you prevent it?

XIDs are 32-bit and compared circularly, so after ~2 billion transactions old xids would look like future ones and rows would disappear. VACUUM prevents it by freezing old tuples. Autovacuum launches anti-wraparound vacuums at `autovacuum_freeze_max_age`; if it falls too far behind, Postgres stops accepting writes. Prevent it by keeping autovacuum healthy, avoiding long transactions, and monitoring `age(datfrozenxid)` and `age(relfrozenxid)`.

Q25. What is the WAL and how does it guarantee durability?

The write-ahead log records every change before the changed data page may be written to disk. A commit is durable once its WAL record is flushed with fsync; data pages can be written lazily. After a crash, Postgres replays WAL from the last checkpoint to rebuild any page changes that never reached the data files. The same WAL stream feeds replication and point-in-time recovery.

Q26. What is a checkpoint and why can it cause latency spikes?

A checkpoint writes all dirty buffers to the data files and records a redo point, bounding recovery time. Flushing many pages at once floods I/O, and the first write to each page after a checkpoint is a full-page image in WAL. Spread it with `checkpoint_completion_target = 0.9` and size `max_wal_size` so checkpoints are triggered by time, not WAL volume. "checkpoints are occurring too frequently" in the log means `max_wal_size` is too small.

Q27. Explain Postgres row-level lock modes.

Four modes, weakest to strongest: `FOR KEY SHARE` (taken by foreign-key checks), `FOR SHARE`, `FOR NO KEY UPDATE` (taken by a normal UPDATE), `FOR UPDATE` (DELETE, key changes, explicit). The split between "key" and "no key" lets a child-table insert (which takes KEY SHARE on the parent) proceed while someone updates non-key columns of the parent. Row locks live in tuple headers, so there is no lock escalation in Postgres.

Q28. What does `SELECT ... FOR UPDATE SKIP LOCKED` do and why is it good for queues?

It locks the selected rows and silently skips rows already locked by other transactions instead of waiting. Several workers can each run "give me the next N pending jobs" and get disjoint sets without blocking, which turns a table into a safe concurrent work queue. Combine it with a partial index on the pending status and `ORDER BY` for fairness. `NOWAIT` instead raises an error on a locked row.

> **RefinePoint:** `OutboxRelay` selects `... WHERE sent_at IS NULL ORDER BY seq LIMIT 100 FOR UPDATE SKIP LOCKED`, and run-service, job-service, the export worker and the datasource refresh worker claim work the same way. We scale by starting more instances — no leader election, no ZooKeeper, no extra broker.

Q29. What is the lock-queue problem with DDL?

An `ALTER TABLE` needs ACCESS EXCLUSIVE. If a long query holds even ACCESS SHARE, the ALTER waits — and every later query on that table waits behind the ALTER in the lock queue. A trivial DDL turns into a total outage

for that table. Use `lock_timeout` (a few seconds) and retry, and run DDL when no long transactions are open.

Q30. How does Postgres detect deadlocks, and how do you avoid them?

After waiting `deadlock_timeout` (1 s by default) a backend checks the wait-for graph; if it finds a cycle, it aborts one transaction with SQLSTATE `40P01`. Avoid them by acquiring locks in a consistent order (sort ids before updating), keeping transactions short, and locking up front with `SELECT ... ORDER BY id FOR UPDATE`. The application should retry deadlock victims.

Q31. What are advisory locks and when would you use them?

Application-defined locks on a 64-bit number that Postgres enforces but attaches no meaning to. Use them for "only one at a time" logic that does not map to a row: serialising a scheduler, generating per-entity sequence numbers, idempotent provisioning. Prefer transaction-scoped `pg_advisory_xact_lock` — it is released automatically and works through transaction-mode pgBouncer. Hash strings to keys with `hashtext()` and accept the tiny collision risk, or use the two-int form to namespace.

Q32. What isolation level does Postgres use by default, and what anomalies are possible?

Read Committed. Each statement sees data committed before it began, so a transaction can see different values on re-read (non-repeatable read), new rows (phantoms), and can lose updates in read-modify-write code unless it uses `SELECT FOR UPDATE`, atomic `UPDATE ... SET x = x + 1`, or optimistic version checks.

Q33. How is Repeatable Read in Postgres different from the SQL standard?

Postgres implements it as snapshot isolation: one snapshot for the whole transaction, so it prevents phantoms as well (the standard allows them at this level). A concurrent update to a row you then try to update raises `40001` instead of overwriting. It still allows write skew. MySQL InnoDB's Repeatable Read is also snapshot-based for plain reads but uses gap/next-key locks for locking reads.

Q34. What is write skew and how does SSI prevent it?

Two transactions read overlapping data, each makes a decision based on what it read and writes disjoint rows, so together they violate an invariant neither violated alone (both on-call doctors go off call). SSI tracks what each serializable transaction read with non-blocking SIREAD locks; when it detects a pattern of read-write dependencies that could form a cycle, it aborts one transaction with `40001`. The application must retry. Alternatives at lower isolation: lock the rows that define the invariant with `FOR UPDATE`, or materialise the conflict into a single row.

Q35. How do you size a HikariCP pool?

Start from the database, not the app: about `(cores × 2) + effective spindles` active connections in total for the DB server, then divide across app instances. Use a fixed-size pool, a short `connection-timeout` so overload fails fast, `max-lifetime` below network idle cut-offs, and leak detection in non-prod. Increase only if measurements show threads waiting for connections while the DB is not saturated. Bigger pools often make latency worse.

Q36. Why is a bigger connection pool not always faster?

Each Postgres connection is a process; once active connections exceed CPU cores and disk parallelism, they compete for CPU, locks and buffers, adding context switching and contention, so throughput flattens and latency rises. Queuing requests in the application (waiting for a connection) is cheaper than queuing inside the database.

Q37. What is pgBouncer transaction mode and what breaks?

pgBouncer gives a client a server connection only for the duration of a transaction, so thousands of app connections share a few dozen DB connections. Anything that relies on session state breaks: `SET` without `LOCAL`, session advisory locks, `LISTEN/NOTIFY`, temp tables, `WITH HOLD` cursors, and server-side prepared statements (fixed in pgBouncer 1.21+ with `max_prepared_statements`, or disable them in JDBC with `prepareThreshold=0`).

Q38. Walk me through how you troubleshoot a slow endpoint caused by the database.

1. Confirm the time is in the DB (tracing spans or logs). 2) Find the statement in `pg_stat_statements` by total time. 3) Check `pg_stat_activity` for lock waits. 4) Get a real plan with `EXPLAIN (ANALYZE, BUFFERS)` using real parameters, or from `auto_explain`. 5) Compare estimates with actuals; fix statistics, add or reshape an index, rewrite the query (range instead of function, `EXISTS` instead of `IN` with large sets, keyset instead of `OFFSET` pagination). 6) Deploy and confirm the improvement in `pg_stat_statements`.

Q39. Why is `OFFSET` pagination slow and what is the alternative?

`OFFSET 100000 LIMIT 20` makes the database produce and discard 100,000 rows. Keyset (seek) pagination remembers the last row's sort key and asks for the next page with `WHERE (started_at, id) < (?, ?) ORDER BY started_at DESC, id DESC LIMIT 20`, which uses the index directly and stays constant-time at any depth. The trade-off: no "jump to page 500".

Q40. When would you partition a table, and what are the pitfalls?

When a table is very large and access or retention follow a key — typically time (keep 13 months, drop older) or tenant hash. Gains: partition pruning, instant retention via `DROP/DETACH`, smaller vacuums and indexes. Pitfalls: PK and unique constraints must include the partition key; queries that do not filter on the key scan all partitions; too many partitions slow planning; foreign keys referencing partitioned tables need PG12+; you must create future partitions ahead of time (or use `pg_partman`).

Q41. Partitioning vs sharding?

Partitioning splits a table inside one database server — same transactions, same queries, the planner handles routing. Sharding splits data across multiple servers — gives horizontal write scaling but loses cross-shard joins, transactions and global constraints unless a layer such as Citus or the application handles them. Partition first; shard only when one primary truly cannot handle writes or storage.

Q42. Streaming vs logical replication?

Streaming replication ships the WAL byte stream; the standby is an exact read-only copy of the whole cluster, same major version — used for HA and read scaling. Logical replication decodes WAL into row changes per table and applies them on a writable subscriber, across versions — used for zero-downtime upgrades, partial replication and change data capture (Debezium).

Q43. How do you handle replica lag and read-your-writes?

Accept eventual consistency for reads that tolerate it (dashboards, reports) and route the rest to the primary: after a user writes, pin their reads to the primary for a short window; or remember the commit LSN and only read from a replica that has replayed it; or keep a read in the same transaction as the write. Monitor `replay_lag` and alert, and have the router stop using a replica that falls too far behind.

Q44. Explain consistent hashing.

Nodes and keys are hashed onto a ring; each key belongs to the first node clockwise. When a node joins or leaves, only the keys in its arc move (~1/N), unlike `hash % N` where changing N remaps almost everything. Virtual nodes (many points per node) balance load and let bigger machines take more points. Used by Cassandra, DynamoDB, CDNs and client-side cache sharding. Redis Cluster is the well-known exception: it uses 16,384 fixed hash slots instead of a ring.

Q45. Explain CAP and PACELC with real systems.

CAP: during a network partition you can keep consistency (reject requests on the minority side) or availability (serve possibly stale data), not both. PACELC adds that even without partitions you trade latency against consistency. Postgres with synchronous replication: PC/EC (consistent, slower commits). Cassandra with `ONE` consistency: PA/EL. Many systems are tunable per request (Cassandra quorum, MongoDB write/read concern).

Q46. Row store vs column store — why is a columnar engine faster for analytics?

Analytics reads a few columns over many rows. A column store reads only those columns, compresses each column well (similar values together: dictionary, run-length, bit-packing), skips blocks using min/max zone maps, and processes values in vectors with SIMD. A row store must read entire rows and evaluate tuple by tuple. Row stores win for point lookups and many small transactional writes.

Q47. Why does RefinePoint use DuckDB for matching instead of doing it in Postgres?

Matching is an analytical workload: load two large files, normalise, join on keys with tolerances across multiple passes, aggregate breaks. DuckDB runs in-process inside the run worker, columnar and vectorised, so a large match needs no network round trips and does not load the shared OLTP database that serves every tenant's UI. Rules written in our expression language are compiled to DuckDB SQL by `SqlCompiler`. Postgres keeps what it is good at: transactional state — processes, runs, breaks, approvals — and the run queue.

Q48. How do you add a NOT NULL column to a 50-million-row table without downtime?

Add it nullable (or with a constant default — PG11+ stores it in the catalog, no rewrite), deploy code that writes it, backfill old rows in small committed batches, add `CHECK (col IS NOT NULL) NOT VALID`, `VALIDATE CONSTRAINT` (only SHARE UPDATE EXCLUSIVE), then `ALTER COLUMN SET NOT NULL`, which on PG12+ trusts the validated check instead of scanning. Use `lock_timeout` for each step.

Q49. How do you rename a column used by a running application?

Expand/contract: add the new column, deploy code writing both, backfill, deploy code reading the new one, stop writing the old one, then drop it in a later release. A direct `RENAME` breaks every running instance of the old version during a rolling deploy. The same applies to type changes and table splits.

Q50. How do you run `CREATE INDEX CONCURRENTLY` with Flyway?

Put it alone in its own migration. It cannot run inside a transaction block; Flyway recognises it for PostgreSQL and executes it outside a transaction, but a migration mixing it with transactional statements fails unless mixed mode is enabled — keeping it alone avoids that. If it fails midway it leaves an `INVALID` index that must be dropped before retrying. It takes longer than a normal build (two table scans) but does not block writes.

Q51. What are your Flyway rules in a team?

Never edit an applied migration (checksum validation will fail and environments diverge); every change is a new version; migrations must be backward compatible with the currently running app version (expand/contract); data backfills in batches, not in a single statement; review migrations for locks like code; and each microservice owns its own schema history. In RefinePoint each service had its history squashed to one `V1` baseline before release 1; everything since is `V2+`.

Q52. How should indexes be designed for a multi-tenant shared-schema database?

Lead every composite index with the tenant id: `(org_id, ...)`. Every query filters by tenant, so the index range is one tenant's rows only; unique constraints become per tenant (`(org_id, name)`); and a query that forgets the tenant filter stands out in the plan. Watch skew — one large tenant can need a different plan than many small ones — and consider hash partitioning by tenant or moving very large tenants to their own database later.

Q53. Row-Level Security — would you use it for multi-tenancy?

RLS enforces tenant isolation inside Postgres with policies like `USING (org_id = current_setting('app.org_id')::uuid)`, so a bug that forgets the filter cannot leak data. Costs: every connection must set the tenant variable per transaction (`SET LOCAL` works with pgBouncer), policies add planning overhead and can confuse the planner, and table owners bypass RLS unless `FORCE ROW LEVEL SECURITY`. It is a good defense-in-depth layer on top of application-level checks, not a replacement for them.

Q54. What happens when you run `UPDATE accounts SET balance = balance - 100 WHERE id = 1` concurrently in two Read Committed transactions?

The first takes the row lock and updates. The second blocks on the row lock. When the first commits, the second re-reads the newest committed version of that row, re-checks the WHERE, and applies `balance - 100` to the *new* value. Both deductions apply — no lost update, because the computation happens inside the UPDATE. If the application had read the balance in a SELECT, computed in Java and written back a literal, one update would be lost.

Q55. How does MySQL InnoDB MVCC differ from Postgres MVCC?

InnoDB updates rows in place in the clustered index and writes the previous version to the **undo log**; readers walk the undo chain to rebuild the version their read view needs, and a purge thread cleans the undo log. Postgres keeps old versions in the table itself and needs VACUUM. InnoDB therefore has no heap bloat but can suffer a growing undo/history list under long transactions; Postgres has no undo but suffers table and index bloat. InnoDB also uses gap and next-key locks to prevent phantoms in locking reads, which Postgres does not have.

25. Caching, Redis & NoSQL

Caching questions look easy and rarely are. "Add Redis" is the junior answer; the senior answer starts with what you are trying to protect (latency, database load, a slow third party), how stale the data may be, and what happens on the day the cache is empty. This chapter covers the patterns, the failure modes that take down production, Redis internals, and when to leave relational databases altogether.

> **RefinePoint:** we deliberately run **without Redis** today. Short-lived answers from other services live in an in-memory `TtlCache`, rate limits live in per-instance buckets, and locks and queues live in PostgreSQL. Being able to explain *why that is enough for now* and *what would make us add Redis* is a stronger answer than listing Redis commands.

25.1 Why cache, and where

A cache trades **memory and staleness** for **latency and load**. It pays off when reads outnumber writes, the same keys repeat (skewed access), the origin is slow or expensive, and slightly old data is acceptable. Measure with the **hit ratio**, but remember what matters is the effect on latency: average latency = `hit% × cache time + miss% × (origin time + cache time)`.



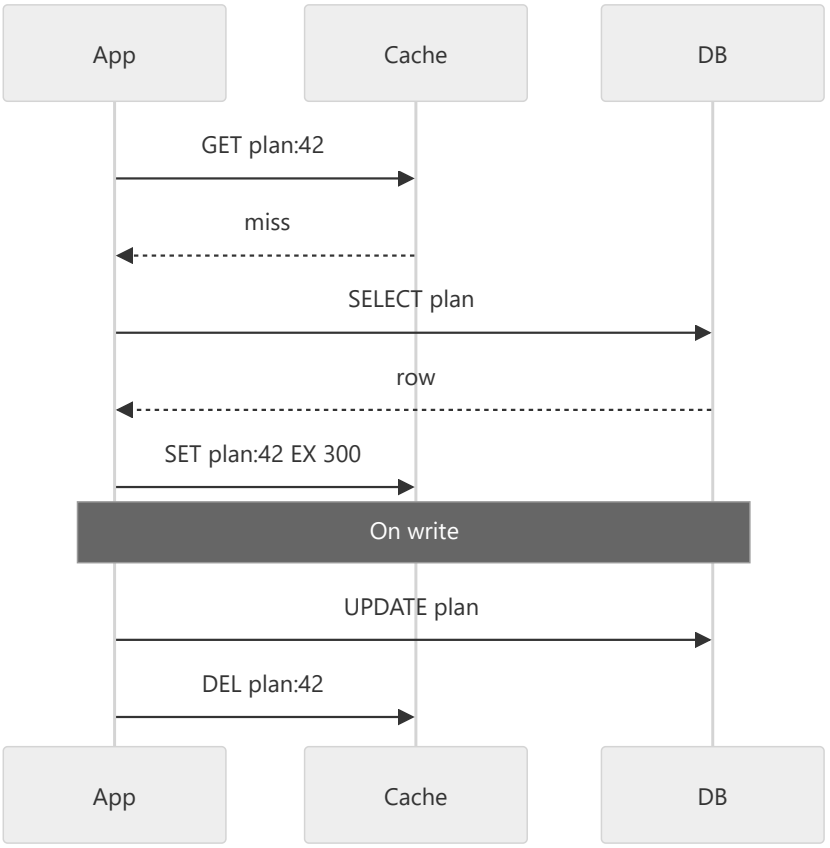
Level	Latency	Scope	Invalidation	Good for
Browser	0 ms	One user	`max-age`, `ETag` revalidation	Static assets, per-user GETs
CDN	~10 ms	Everyone	TTL, purge API, versioned URLs	JS bundles, images, public pages
Gateway	~1 ms	All callers	TTL	Public, identical responses
App local	~100 ns	One instance	TTL, broadcast	Config, permissions, reference data
Redis	~1 ms	All instances	Explicit delete, TTL	Sessions, computed views, counters
DB buffers	Automatic	Everyone	Automatic	Everything, for free

Per-tenant data behind authentication should almost never reach a shared CDN or gateway cache: a cache key that forgets the tenant or the user is a data leak, not a performance bug.

25.2 Caching patterns

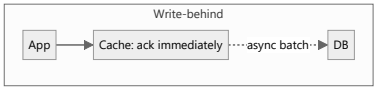
Cache-aside (lazy loading)

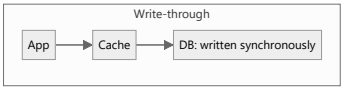
The application owns the logic: read the cache, on miss read the DB and populate; on write update the DB and **delete** the key. This is the default pattern.

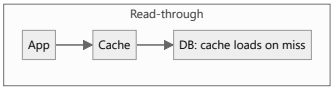


Why delete rather than update on write? Two concurrent writers can apply cache updates in the opposite order to their DB commits, leaving a stale value indefinitely. Deleting converges on the next read.

Read-through, write-through, write-behind, refresh-ahead







Pattern	Who talks to DB	Read latency	Write latency	Consistency	Risk
Cache-aside	App	Miss is slow	Normal	Short staleness window	Stampede on miss
Read-through	Cache library	Miss is slow	Normal	Same	Loader tied to cache
Write-through	Cache synchronously	Fast (always warm)	Slower (two writes)	Strong-ish	Caches never-read data
Write-behind	Cache asynchronously	Fast	Very fast	Eventual	**Data loss** if the cache dies
Refresh-ahead	Cache before expiry	Fast	Normal	Fresher	Wasted refreshes

Caffeine's `LoadingCache` is read-through; `refreshAfterWrite` is refresh-ahead (serve the old value, reload in the background). Write-behind suits counters and metrics, never money.

Eviction policies

Policy	Evicts	Weakness
LRU	Least recently used	One full scan flushes useful entries
LFU	Least frequently used	Old popular entries linger unless counts decay
FIFO	Oldest inserted	Ignores usage
TTL	Expired entries	Not a capacity policy on its own
W-TinyLFU (Caffeine)	Admission by frequency sketch + LRU windows	Near-optimal hit rate, more complex
Random	Any	Surprisingly decent, very cheap

A TTL is about **freshness**; eviction is about **capacity**. You usually need both. Redis offers `maxmemory-policy`: `noeviction` (writes fail, the default), `allkeys-lru`, `allkeys-lfu`, `volatile-lru`, `volatile-ttl`, `allkeys-random`. Redis LRU is **approximate**: it samples a few keys (`maxmemory-samples`, default 5) rather than keeping a global list.

25.3 Spring Cache abstraction

Spring wraps a bean in a proxy; `CacheInterceptor` checks the `Cache` for the computed key before calling the method.

```
@Configuration @EnableCaching
class CacheConfig {
    @Bean CacheManager cacheManager() {
        var mgr = new CaffeineCacheManager("plans", "rates");
        mgr.setCaffeine(Caffeine.newBuilder().maximumSize(10_000)
            .expireAfterWrite(Duration.ofMinutes(5)).recordStats());
        return mgr;
    }
}

@Service
class PlanService {
    @Cacheable(cacheNames = "plans", key = "#orgId", unless = "#result == null", sync = true)
    public Plan planOf(UUID orgId) { return repo.findPlan(orgId); }

    @CachePut(cacheNames = "plans", key = "#orgId")
    public Plan change(UUID orgId, String code) { return repo.save(orgId, code); }

    @CacheEvict(cacheNames = "plans", key = "#orgId")
    public void reset(UUID orgId) { repo.reset(orgId); }
}
```

Annotation	Effect
`@Cacheable`	Skip the method on a hit, store the result on a miss
`@CachePut`	Always run the method, store the result
`@CacheEvict`	Remove an entry (`allEntries = true` ; `beforeInvocation = true` to evict even if the method throws)
`@Caching`	Group several of the above

Attributes: `key` (SpEL), `condition` (evaluated **before** the call, decides whether to use the cache at all), `unless` (evaluated **after**, can see `#result`, vetoes storing), `sync = true` (only one thread loads a given key per JVM, if the provider supports it).

Proxy pitfalls:

1. **Self-invocation** bypasses the proxy: `this.planOf(id)` inside the same class is never cached.
2. Only `public` methods on Spring-managed beans are intercepted by default.
3. Default key is built from all parameters (`SimpleKey`), so an unrelated parameter fragments the cache and a **missing tenant in the key leaks data across tenants**.
4. `@CacheEvict` runs *after* the method by default, but the surrounding transaction may not have committed yet. Another request can re-cache the old row between the evict and the commit. Evict after commit (`TransactionAwareCacheManagerProxy` , `@TransactionalEventListener(phase = AFTER_COMMIT)`).
5. Mutable cached objects: Caffeine returns the same instance by reference, so a caller's mutation changes everyone's value. Cache immutable records.
6. Redis serialisation: a class change can make old entries undeserialisable. Version the key prefix or use JSON with tolerant reading.

25.4 Local (Caffeine) vs distributed (Redis)

	Caffeine	Redis
Latency	Nanoseconds, in the heap	Network hop, ~1 ms
Consistency across instances	None; each has its own copy	Shared
Survives restart	No	Optional
Capacity	Limited by heap, adds GC pressure	Its own server, GBs–TBs
Serialisation	None	Every read and write
Failure mode	Cannot fail independently	New dependency, needs fallback

A **two-level** cache uses both: Caffeine with a very short TTL (seconds) in front of Redis absorbs hot keys. Invalidate across instances with Redis pub/sub or simply accept the short TTL.

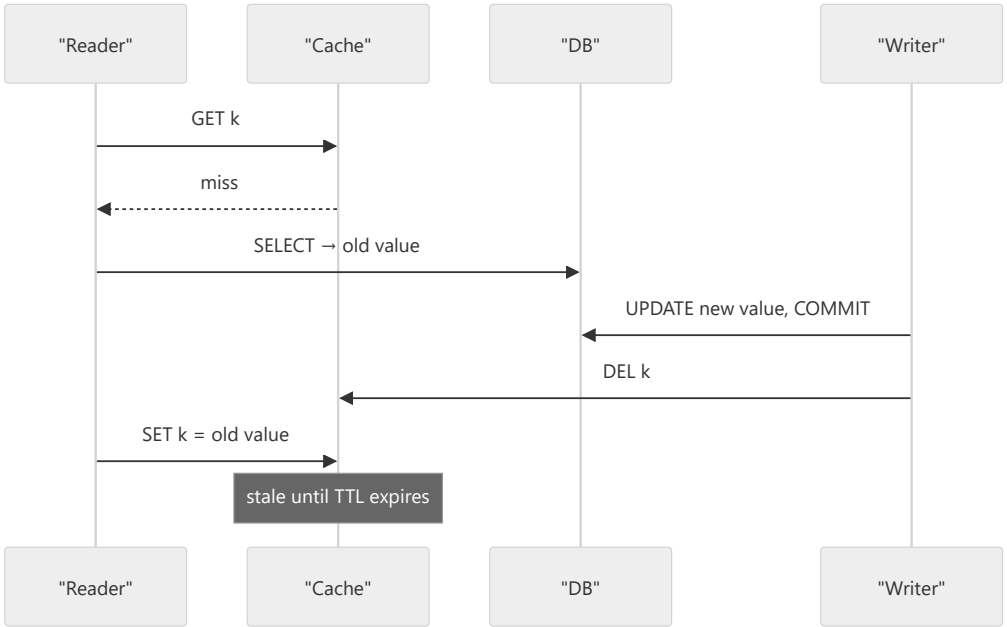
RefinePoint: every service verifies a caller's token itself, but workspace roles live in identity-service. `RemoteWorkspaceAccess` asks identity over `InternalClient` and caches answers in `TtlCache` for `rp.cache-ttl` (30 seconds by default). Without it, every request would make an extra HTTP call. The accepted cost is written down: a role change or revoked API key takes effect everywhere within that TTL. `RemoteApiKeyVerifier` shows a good detail: rejected keys are cached separately for only 5 seconds, so a bad key cannot hammer identity, but a newly created key works almost at once. `TtlCache` is a small `ConcurrentHashMap` with expiry that simply clears itself when full — crude, but "only costs a few extra calls" and needs no dependency.

25.5 Invalidation and consistency

"There are only two hard things..." — invalidation is hard because the DB write and the cache delete are **two separate systems with no shared transaction**.

Strategy	How	Staleness	Cost
TTL only	Let it expire	Up to the TTL	Simplest; choose TTL from business tolerance
Delete on write	App deletes after commit	Small race	App must know every affected key
Versioned keys	`report:v17:42`; bump the version	None for new reads	Old keys age out via TTL
Event-driven	Consume change events, delete keys	Event lag	Needs reliable events
CDC	Debezium reads the WAL, deletes keys	Lag, but catches every writer	Infrastructure

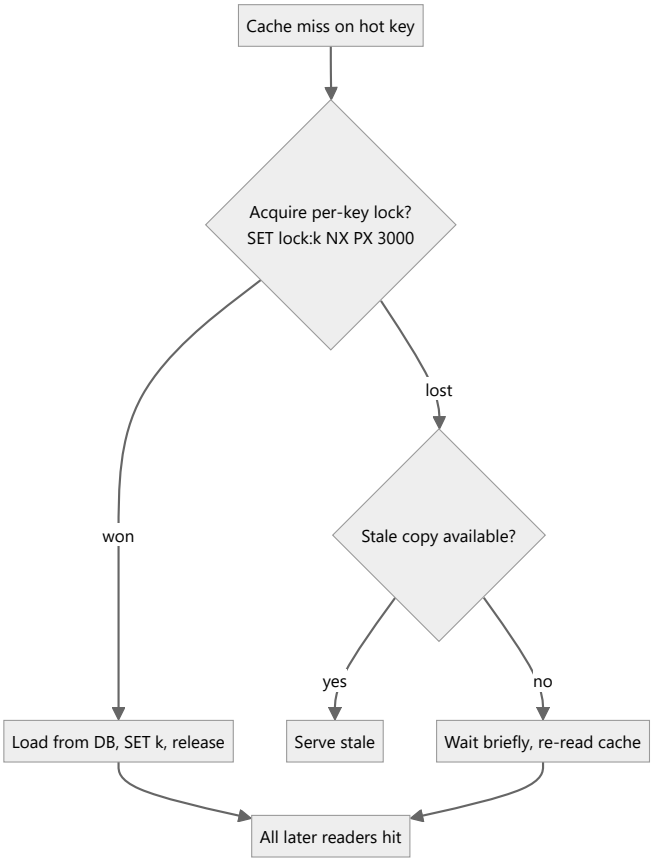
The classic race with cache-aside:



Mitigations: **always set a TTL** as the backstop; **delayed double delete** (delete, then delete again after a short delay); write with a **version** and refuse to overwrite a newer one (a Lua compare-and-set). Do not promise strong consistency from a cache. If a value must be correct at decision time (a balance, a plan limit that blocks a purchase), read the source of truth for the decision and use the cache for display.

25.6 Failure modes: stampede, penetration, avalanche, hot keys

Problem	What happens	Fixes
Stampede (thundering herd, dogpile)	A hot key expires; thousands of requests all miss and hit the DB	Per-key lock or request coalescing, stale-while-revalidate, early refresh
Penetration	Requests for keys that do not exist always miss and reach the DB (often an attack)	Cache nulls briefly, Bloom filter, validate input
Avalanche	Many keys expire together, or the cache goes down, flattening the DB	Jittered TTLs, circuit breaker and limiter in front of the DB, HA cache, warm-up
Hot key	One key takes huge traffic, saturating one Redis node	Local L1 cache, split key (`k:1..k:N`), read from replicas
Big key	A multi-MB value blocks the single thread	Split into a hash, compress, `UNLINK` instead of `DEL`



Request coalescing in one JVM: `@Cacheable(sync = true)`, or Caffeine's `get(key, loader)`, which guarantees one load per key. Across JVMs use a short Redis lock.

Jittered TTL: `ttl = base + random(0, base × 0.1)` so entries loaded together do not expire together.

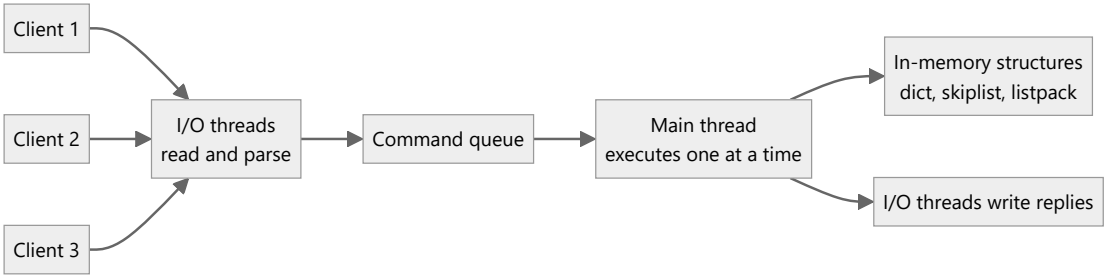
Probabilistic early expiry (XFetch): each reader may recompute early, with probability rising near expiry: recompute if `now - delta × beta × ln(rand()) >= expiry`, where `delta` is how long the value takes to compute. No locks; slow-to-compute keys refresh earlier.

> **Trap:** `TtlCache.get` in RefinePoint has no per-key coalescing: two concurrent misses both call the loader. That is fine because the origin is a cheap indexed lookup and the instance count is small. On an expensive origin the same design would stampede; use `ConcurrentHashMap.computeIfAbsent` with a future, or Caffeine.

Bloom filter: a bit array with k hash functions. Adding a key sets k bits; a lookup answers **"definitely not present"** or **"maybe present"** — false positives possible, false negatives never. About 10 bits per key gives ~1% false positives. Put it in front of the cache to reject impossible ids. Deletes need a counting variant.

25.7 Redis internals

Redis executes commands on **one main thread** with an event loop (`epoll`), so each command is atomic and there are no data locks. Since Redis 6, optional **I/O threads** handle socket reads, parsing and writes, but commands still execute serially. It is fast because everything is in memory, the structures are efficient and there is no locking. The flip side: **one slow command blocks everybody** (`KEYS *`, `SMEMBERS` on a huge set, `DEL` of a big key). Use `SCAN` and `UNLINK`.



Type	Encoding	Commands	Use cases
String	int, embstr, raw (max 512 MB)	`SET`, `GET`, `INCR`, `SETNX`	Cached JSON, counters, locks
Hash	listpack → hashtable	`HSET`, `HINCRBY`	Objects with partial updates, sessions
List	quicklist	`LPUSH`, `BRPOP`, `LRANGE`	Simple queues, recent items
Set	intset, listpack → hashtable	`SADD`, `SINTER`	Unique members, tags
Sorted set	listpack → skiplist + hash	`ZADD`, `ZRANGE`, `ZRANGEBYSCORE`	Leaderboards, sliding windows, delayed jobs
Stream	Radix tree of listpacks	`XADD`, `XREADGROUP`, `XACK`	Durable log with consumer groups
HyperLogLog	Sparse/dense registers (12 KB)	`PFADD`, `PFCOUNT`	Approximate distinct count, ~0.81% error
Bitmap	String of bits	`SETBIT`, `BITCOUNT`	Daily active users, feature flags
Geo	Sorted set of geohashes	`GEOADD`, `GEOSEARCH`	Nearby search

```
ZADD leaderboard 1500 "alice"            # leaderboard: score + member
ZREVRANGE leaderboard 0 9 WITHSCORES     # top 10, O(log n + 10)
ZREVRANK leaderboard "alice"             # rank

PFADD visitors:2026-11-02 "u1" "u2"      # approximate distinct users, 12 KB
SETBIT dau:2026-11-02 12345 1            # bit per user id
```

Why a skiplist for sorted sets? Range operations and rank are straightforward, it uses similar memory to a balanced tree, and it is far simpler to implement and tune than an in-memory B-tree. A hash alongside gives O(1) score lookup.

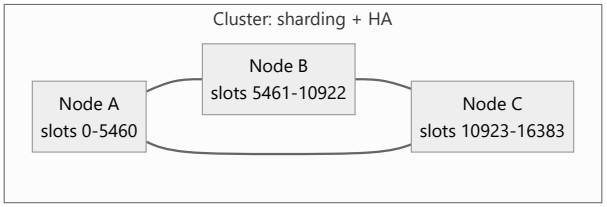
Pipelining batches commands in one round trip (not atomic). `MULTI/EXEC` queues commands and runs them atomically, without rollback. **Lua scripts** (`EVAL`) and **Functions** run atomically on the main thread — so keep them short.

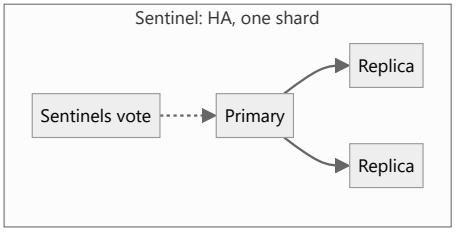
25.8 Persistence, replication and clustering

	RDB	AOF
What	Point-in-time binary snapshot	Log of write commands
How	`fork()` , child writes; copy-on-write	Append; `fsync` per `appendfsync` policy
Data loss	Since the last snapshot (minutes)	`everysec` : ~1 s; `always` : ~none, slow
Restart speed	Fast	Slower; compacted by rewrite
Cost	Fork latency, CoW memory spike on big heaps	Larger files, more I/O

Use both in most persistent setups (Redis 7's AOF is a multi-part base + increments). A pure cache can disable persistence, accepting a cold start — plan for that stampede.

Replication is asynchronous: the replica does a full sync (an RDB) and then applies a stream of commands. `WAIT n ms` narrows the loss window but does not make Redis strongly consistent. A failover can lose acknowledged writes.





	Sentinel	Cluster
Goal	Automatic failover	Horizontal scaling + failover
Data	All on one primary	Split over **16,384 hash slots**: `CRC16(key) % 16384`
Client	Asks Sentinel for the primary	Slot map; follows `MOVED` / `ASK` redirects
Multi-key ops	Unrestricted	Only within a slot; force with **hash tags** `{org42}:a` , `{org42}:b`
Quorum	Sentinels (odd, ≥ 3)	Primaries vote via gossip bus

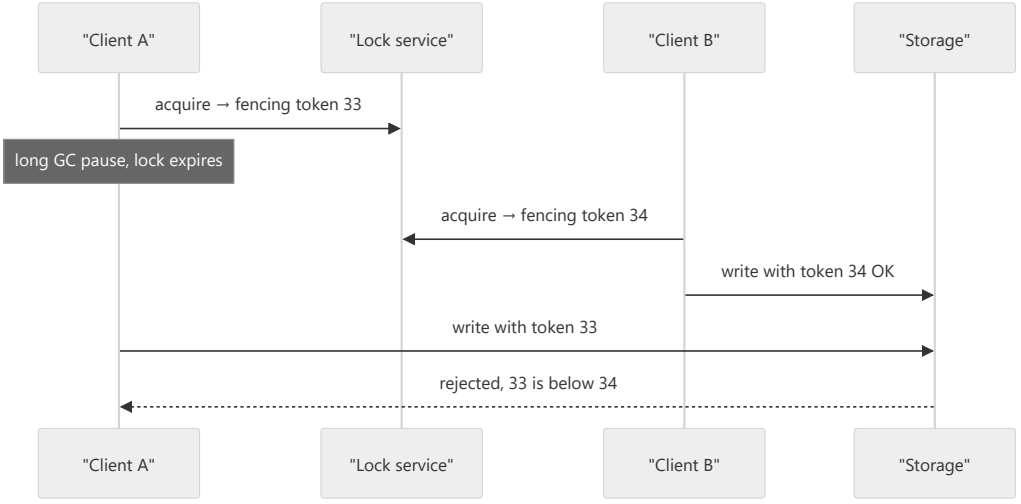
25.9 Distributed locks

The simple lock: set a key only if absent, with an expiry and a unique token; release only if you still own it.

```
SET lock:report:42 "9f3a-uuid" NX PX 30000   # acquire
```

```
-- release atomically: never delete someone else's lock
if redis.call('GET', KEYS[1]) == ARGV[1] then return redis.call('DEL', KEYS[1]) else return 0 end
```

The failure no Redis setting can fix: the holder pauses (a GC pause, a network stall), the lock expires, someone else acquires it, and the first holder wakes up still believing it holds the lock.



Fencing tokens solve it: the lock service issues a monotonically increasing number, and the *storage* rejects anything lower. Plain Redis has no such token (though `INCR` can issue one), and the storage must cooperate.

The Redlock debate. Redlock acquires the lock on a majority of N independent Redis primaries. Martin Kleppmann argued that it relies on timing assumptions (bounded clock drift and pauses) and gives no fencing, so it is unsafe when correctness depends on the lock. Salvatore Sanfilippo replied that the assumptions are reasonable in practice and the clock issues can be handled operationally. The practical conclusion: for **efficiency** (avoiding duplicate work, where a rare double run is harmless) a single-instance `SET NX PX` is enough; for **correctness**, use a consensus store (etcd, ZooKeeper) with fencing, or the transactional database that holds the data.

	Redis `SET NX PX`	Postgres row lock / `SKIP LOCKED`	Postgres advisory lock
Released on crash	After TTL	Immediately, with the transaction	Immediately
Scope	Any key	A row	Any 64-bit key
Holder pause	Lock can expire under it	Held while the transaction lives	Held while the transaction lives
Atomic with your data	No	**Yes**	**Yes** (transaction-scoped)
Throughput	Very high	Lower; holds a connection	Lower; holds a connection

> **RefinePoint:** we chose Postgres. Work queues claim with `FOR UPDATE SKIP LOCKED`, and invariants that do not map to one row use `pg_advisory_xact_lock(hashtext(...))` — the run-queue claim, run numbers per process, workspace provisioning. A claim and the status change commit together, so the lock cannot disagree with the data, and a crashed worker's lock vanishes with its connection. The database is already a required dependency, and a claim takes milliseconds; a second system would add a failure mode without fixing a problem we have. A long-running run is not held under a lock: it is marked RUNNING with an instance name and a `heartbeat_at`, and a reaper re-queues runs whose instance stopped reporting.

25.10 Rate limiting

Algorithm	Idea	Accuracy	Memory	Burst behaviour
Fixed window	Counter per time slot	Poor at boundaries (2× burst)	O(1)	Spike at the edge
Sliding log	Timestamps in a window	Exact	O(requests)	Exact
Sliding counter	Weighted previous + current window	Close	O(1)	Smooth
Token bucket	Refill rate, burst capacity	Good	O(1)	Allows bursts up to capacity
Leaky bucket	Constant drain queue	Good	O(1)	Smooths out bursts

Redis versions must be **atomic**, otherwise the read and the write race:

```
# Fixed window: INCR then EXPIRE on first hit (put both in Lua or MULTI)
# Sliding log with a sorted set
ZREMRANGEBYSCORE rl:u1 0 (now-60000)
ZCARD rl:u1
ZADD rl:u1 now now
PEXPIRE rl:u1 60000
```

```
-- Token bucket: KEYS[1]=bucket, ARGV: capacity, refill per ms, now, requested
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local cap, rate, now, want = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3]),
tonumber(ARGV[4])
local tokens = tonumber(b[1]) or cap
local ts = tonumber(b[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)
local ok = tokens >= want
if ok then tokens = tokens - want end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate))
return ok and 1 or 0
```

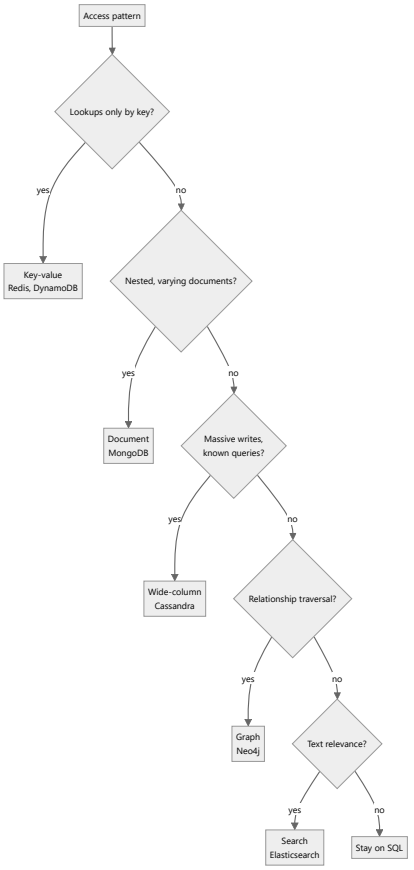
RefinePoint: two per-instance implementations. `RateLimitFilter` in api-gateway is a token bucket in a `ConcurrentHashMap`: 300 requests a minute per API key, 900 per signed-in user, 1,800 per address, with 429, `Retry-After` and `X-RateLimit-*` headers, and idle buckets forgotten periodically. `SlidingWindowLimiter` in `ryfn-platform` is a sliding log (`ArrayDeque` of timestamps per key) guarding expensive or public actions: AI usage per user, the public assistant and try-import per IP, SSO lookups, public confirmation links. Neither is shared between instances, so with N instances the real limit can reach N times the configured one. That is acceptable for now because the limits protect capacity rather than meter money (billing limits are enforced by `PlanLimits` from the database), the instance count is small, and there is no extra network hop or Redis outage to plan for. When we run many gateway instances or need an exact per-customer quota, these move to Redis with the Lua script above and the interface stays the same.

25.11 Sessions and pub/sub

Session storage: a stateless JWT needs no shared session, at the cost of hard revocation. Server-side sessions in Redis (Spring Session) give instant logout and small cookies, at the cost of a lookup per request. Use a hash with a TTL, slide the expiry on access, and make Redis highly available because losing it logs everyone out.

Pub/sub is fire-and-forget: a subscriber that is down misses the message, and there is no persistence or replay. It suits L1 cache invalidation and live notifications. Redis **Streams** add persistence, consumer groups and acknowledgements; Kafka is still the choice for high-volume, long-retention, partitioned event logs.

25.12 NoSQL families



Family	Data model	Strength	Weakness	Examples
Key-value	Opaque value by key	Speed, simple scaling	No query by value	Redis, DynamoDB
Document	JSON-like, nested	Flexible schema, aggregates	Joins, cross-document consistency	MongoDB, Couchbase
Wide-column	Partition key → sorted rows	Write throughput, linear scale	Design per query, no joins	Cassandra, ScyllaDB, HBase
Graph	Nodes + edges	Multi-hop traversal	Sharding is hard	Neo4j, Neptune
Search	Inverted index	Relevance, facets, fuzzy	Not a system of record	Elasticsearch, OpenSearch
Time-series	Time-partitioned, compressed	Ingest + rollups, retention	Narrow use	InfluxDB, TimescaleDB, Prometheus

Choose SQL when you need transactions across entities, flexible ad-hoc queries, joins and constraints, and the data fits one primary. **Choose NoSQL** when a specific access pattern or scale requirement justifies giving those up. Postgres narrows the gap: `jsonb` with GIN handles many document needs, `pg_trgm` and full-text search handle moderate search, and a partial index handles queues.

MongoDB data modelling basics

Model for how the data is **read**: data read together stays together. **Embed** for owned, bounded children; **reference** for shared, independently-updated or unbounded ones. Documents cap at 16 MB, so unbounded arrays are the classic anti-pattern (use the *bucket* pattern). Single-document writes are atomic; multi-document transactions exist since 4.0 but cost more, so needing them often signals a wrong model. Scaling is by **replica sets** (primary + secondaries) and **shard keys** — pick a high-cardinality, non-monotonic key, as resharding is expensive.

```
{ "_id": "o1", "customer": {"id": "c9", "name": "Asha"},
  "items": [ {"sku": "A1", "qty": 2, "price": 450} ],
  "status": "PAID" }
```

Cassandra in one paragraph

Data is placed on a consistent-hash ring by **partition key**, and rows inside a partition are sorted by **clustering key**. Writes go to a commit log + memtable, flush to immutable **SSTables**, and merge via compaction (an **LSM tree**), so writes are very fast and reads may touch several files (Bloom filters help). Consistency is tunable per request: `R + W > N` gives read-your-writes. Tables are designed **per query**, with deliberate denormalisation.

Elasticsearch and the inverted index

```
Documents                       Inverted index (after analysis)
 1: "Bank fee reversed"         bank     → [1, 3]
 2: "Ledger fee"                fee      → [1, 2]
 3: "Bank statement"            ledger   → [2]
                                reversed → [1]
                                statement→ [3]
Query "bank fee": union of postings, scored by BM25
```

An **analyzer** (character filters, tokenizer, token filters such as lowercase and stemming) turns text into terms; each term points to a sorted **posting list** of documents. Lookups are near-O(1) per term rather than a scan, unlike `LIKE '%x%'`. An index is split into **shards** (Lucene indexes), each with immutable **segments**; a `refresh` (default 1 s) makes new documents searchable, hence "near real-time", and segment merges reclaim deletes. `text` fields are analysed for search; `keyword` fields are exact and sortable. Treat it as a derived read model: keep the truth in the database and feed Elasticsearch via events or CDC.

25.13 Interview questions and answers

Q1. When should you NOT add a cache?

When the data changes as often as it is read, each key is rarely repeated, the origin is already fast enough, or the staleness cannot be tolerated. Fix the slow query first: a cache in front of a missing index hides the problem until the first cold start. Every cache adds invalidation logic, a failure mode and a capacity plan, so it must earn its place with measured numbers.

Q2. What is a good hit ratio?

It depends on the cost of a miss, but read-heavy caches usually aim for over 90%. A low ratio means poor key design (too specific), a TTL that is too short, or a cache that is too small. Moving from 90% to 99% cuts origin load tenfold, so small improvements matter more than they look. Watch evictions alongside hits, and the p99 latency, as those show real impact.

Q3. Cache-aside vs read-through?

Both load on a miss; the difference is who does it. In cache-aside the application code reads the DB and fills the cache; in read-through the cache library calls a loader. Read-through centralises loading and makes per-key coalescing easy (Caffeine `LoadingCache`); cache-aside works with any store and is what Redis deployments usually do because Redis cannot call your database.

Q4. Write-through vs write-behind?

Write-through writes cache and DB synchronously, so the cache is always warm at the cost of slower writes and caching data nobody reads. Write-behind acknowledges after the cache write and flushes asynchronously, giving very fast writes and batching, but a cache failure loses data and ordering becomes your problem. Write-behind is for counters and analytics, not for financial records.

Q5. On a write, should you update the cache or delete the key?

Delete, in almost all cases. Concurrent updates can land in the cache in a different order from the DB commits, leaving a permanently wrong value until the TTL, while a delete converges on the next read. Updating also wastes work when the computed value is expensive or never read again. Update in place only when the write is serialised per key.

Q6. Should you delete the cache entry before or after the DB write?

After the commit. Deleting first leaves a window in which a reader re-caches the old row before the write lands. Deleting afterwards still has a small race (see the reader/writer sequence) and can fail if the process dies between the commit and the delete, so keep a TTL as the backstop. Delayed double delete, or a delete driven from the outbox or CDC, closes most of the remaining gap.

Q7. How do you keep cache and database consistent?

You don't fully — they are two systems without a shared transaction. Choose a staleness bound and engineer to it: TTL on everything, delete after commit, event- or CDC-driven invalidation for writers you do not control, versioned keys for immutable snapshots. For decisions that must be correct, read the source of truth. Say the bound aloud: "a role change takes effect within 30 seconds" is a design statement, not a defect.

Q8. Explain cache stampede and three ways to prevent it.

When a hot key expires, all concurrent requests miss together and hit the origin. Prevent it with: (1) a per-key lock or request coalescing so one loader runs and others wait or get stale data; (2) refresh-ahead or probabilistic early expiry so the key is renewed before it expires; (3) stale-while-revalidate, serving the old value while one background reload runs. Jitter helps when many keys expire together but does not save one very hot key.

Q9. Cache penetration vs breakdown vs avalanche?

Penetration: requests for keys that do not exist bypass the cache every time; fix with null caching and Bloom filters. Breakdown (stampede): one hot key expires under load; fix with locks and early refresh. Avalanche: many keys expire together or the cache fails; fix with jittered TTLs, HA, and rate limiting or circuit breaking in front of the DB.

Q10. How does a Bloom filter work and what are its limits?

A bit array and k hash functions: add sets k bits, lookup checks them, and any zero means definitely absent. It can report false positives but never false negatives, and it uses about 10 bits per key for 1% errors. Standard filters cannot delete and must be sized upfront; use a counting or cuckoo filter if deletes matter. RedisBloom provides one server-side.

Q11. What are the risks of caching nulls?

A key created right after caching "not found" stays invisible until the TTL, and an attacker can fill memory with fake keys. Use a short TTL for negatives and bound the number of entries. RefinePoint's `RemoteApiKeyVerifier` uses exactly this split: 5 seconds for rejected keys, longer for valid ones.

Q12. How do you handle a hot key in Redis?

Find it with `redis-cli --hotkeys` (requires an LFU policy) or client metrics. Add a short-TTL local cache in each app instance, which usually eliminates most of the traffic; split the key into `k:1..k:N` and read a random

copy; or read from replicas where staleness is acceptable. A hot key is a single-thread and network problem, so adding cluster nodes does not fix it because a key lives in one slot.

Q13. Explain `@Cacheable`, `@CachePut` and `@CacheEvict`.

`@Cacheable` skips the method on a hit and stores the result on a miss. `@CachePut` always runs the method and stores the return value. `@CacheEvict` removes an entry or, with `allEntries = true`, the whole cache. All are applied by an AOP proxy (`CacheInterceptor`), with keys computed by SpEL or the default `SimpleKeyGenerator`.

Q14. What is the difference between `condition` and `unless`?

`condition` is evaluated before the method and decides whether caching applies at all, so it cannot use `#result`. `unless` is evaluated after and can veto storing, so `unless = "#result == null"` is the common form. With `condition` false, the method simply runs without any cache interaction.

Q15. Why is my `@Cacheable` method not caching?

Most often self-invocation: the call goes through `this`, not the proxy. Other causes: `@EnableCaching` is missing, the method is not public, the object is not a Spring bean, the key differs each call (an entity without `equals`, or a timestamp parameter), `unless` rejects the result, or the TTL is effectively zero. Fix self-invocation by moving the method to another bean.

Q16. What does `@Cacheable(sync = true)` do?

Only one thread computes a missing key per JVM; others wait for the result. It prevents a local stampede but not a stampede across instances, and it cannot be combined with `unless` or multiple caches. The provider must honour it. Caffeine does; `RedisCache` uses a local lock, so it is still per JVM.

Q17. What is the transaction problem with `@CacheEvict`?

The evict can run before the surrounding transaction commits, so another request re-caches the old row just before the new value commits. The cache then holds stale data until the TTL. Evict after commit with `TransactionAwareCacheManagerProxy` or an `@TransactionalEventListener(phase = AFTER_COMMIT)`.

Q18. How would you write cache keys in a multi-tenant system?

Always include the tenant id, plus any input that changes the answer: `plan:{orgId}`, `perm:{orgId}:{userId}`. A key without the tenant is a cross-tenant data leak, the worst bug a SaaS can ship. Add a schema version prefix so a deploy that changes the DTO shape does not read incompatible entries. Hash tags such as `{org42}` keep one tenant's keys on one Redis Cluster slot when multi-key operations are needed.

Q19. Why Caffeine over a `ConcurrentHashMap` with timestamps?

Caffeine provides size bounds with a near-optimal eviction policy (W-TinyLFU), per-key load coalescing, TTLs, async refresh and hit/miss statistics, with low contention. A map with timestamps has none of that: `TtlCache`

in RefinePoint clears everything when full and can run the loader twice. We accept this for small entry counts and cheap origins; on a hotter path I would switch rather than extend it.

Q20. What is W-TinyLFU in simple terms?

A small "window" LRU admits new entries, and a compact frequency sketch decides whether a candidate deserves to replace the eviction victim in the main space. It resists one-time scans (which ruin LRU) and adapts to changing popularity (which ruins pure LFU), and counts decay over time.

Q21. Why is Redis so fast despite being single-threaded?

Everything is in memory, the data structures are tuned, there is no lock contention or context switching, and an event loop multiplexes sockets efficiently. The bottleneck is usually network and memory, not CPU. I/O threads (Redis 6+) help with socket work, and for more CPU you shard rather than add threads.

Q22. If Redis is single-threaded, is every operation atomic?

Each single command is, as are `MULTI/EXEC` blocks and Lua scripts. Sequences you send separately are not: `GET` followed by `SET` can interleave with another client. Use `INCR`, `SET NX`, `HINCRBY` or a Lua script. Note that `MULTI/EXEC` does not roll back: if one command fails at runtime, the others still apply.

Q23. What commands would you avoid in production?

`KEYS`, `FLUSHALL`, `SMEMBERS` / `HGETALL` on huge collections, and `DEL` on big keys, because they block the only execution thread. Use `SCAN` / `SSCAN` / `HSCAN` and `UNLINK`, and rename or disable dangerous commands via ACLs. `SLOWLOG GET` shows what hurt you.

Q24. How would you build a leaderboard and a sliding-window limiter with Redis?

Both use sorted sets. A leaderboard uses `ZINCRBY` to update and `ZREVRANGE` to read, with `ZREVRANK` giving a rank in O(log n). A sliding-window limiter stores each request with its timestamp as score: `ZREMRANGEBYSCORE` drops old entries, `ZCARD` counts, `ZADD` records, all in one Lua script for atomicity. Member values must be unique, or two requests in one millisecond collapse into one entry.

Q25. HyperLogLog vs Set vs bitmap for counting uniques?

A set is exact but its memory grows with cardinality. A bitmap is exact and compact only when ids are small dense integers. HyperLogLog gives ~0.81% error in at most 12 KB for any cardinality and merges with `PFMERGE`, so it suits "unique visitors per day" where approximation is fine, not billing.

Q26. RDB vs AOF: which do you choose?

For a pure cache, often neither or RDB only: it restarts cold but is simple. For data you would miss, enable AOF with `everysec` (lose at most about a second) together with RDB for fast restart and backups. Watch the fork: on a big dataset, RDB and AOF rewrite can cause a latency spike and a copy-on-write memory surge, so leave headroom (and disable transparent huge pages).

Q27. Does Redis lose data? Is it a safe primary database?

It can: replication is asynchronous, so a failover loses acknowledged writes, and AOF `everysec` can lose about a second on crash. `WAIT` reduces this but does not provide strong consistency. It is fine as a primary store only for data you can tolerate losing or can rebuild. Keep ledgers and orders in a transactional database.

Q28. Sentinel vs Cluster?

Sentinel gives automatic failover for one dataset that fits on one primary; Cluster also splits data across primaries with 16,384 hash slots. Choose Sentinel when memory and throughput fit a single node, as it allows multi-key operations and Lua freely. Choose Cluster when you need to scale out, and design keys around slots with hash tags. Cluster does not support multiple databases (only DB 0).

Q29. How do hash slots differ from consistent hashing?

Consistent hashing places keys and nodes on a ring, and membership changes move an arc of keys. Redis Cluster instead maps each key to one of 16,384 fixed slots and assigns slots to nodes, so rebalancing moves explicit slots and clients cache a slot map. This makes resharding explicit and controllable, with `MOVED` and `ASK` redirects handling keys in transit.

Q30. How do you implement a distributed lock with Redis correctly?

`SET key token NX PX ttl` to acquire, a Lua compare-and-delete to release, a TTL longer than the worst-case work (renewed by a watchdog if needed, as Redisson does), and retry with backoff. Then state the limit: expiry under a paused holder means two owners are possible, so a correctness-critical path needs fencing at the resource.

Q31. What is a fencing token?

A number that increases with each lock grant, sent with every write; the resource rejects writes carrying a lower number than one it has already seen. It protects against a paused or partitioned ex-holder. It requires the storage layer to check it, which a plain S3 write or external API cannot do without a conditional mechanism.

Q32. Explain the Redlock controversy.

Redlock takes the lock on a majority of independent Redis nodes within a time limit. Kleppmann's critique is that it depends on clocks and bounded pauses, and offers no fencing, so it is too weak for correctness and too heavy for efficiency. Antirez counters that its timing assumptions are practical and that it checks elapsed time after acquisition. As a practitioner: efficiency locks need one Redis; correctness locks need consensus plus fencing, or a database transaction.

Q33. Why did you use Postgres instead of Redis for locks and queues?

Because the lock must be atomic with the state it protects, and Postgres gives that by construction: claim and status change commit together, and a crash releases everything. Redis would add a dependency, an expiry race, and a two-system consistency problem. The trade-off is throughput and holding a connection during the

lock, which is why our transactions are short and long runs use a heartbeat instead of a held lock. At thousands of claims per second I would revisit it.

> **RefinePoint:** the run queue serialises claims with `pg_advisory_xact_lock(hashtext('run-queue'))` so a per-workspace concurrency cap holds exactly, and picks a row with `FOR UPDATE OF r SKIP LOCKED`. We scale by adding instances, with no leader election and no extra infrastructure.

Q34. Token bucket vs sliding window vs leaky bucket?

Token bucket refills at a steady rate and allows bursts up to capacity, which is friendly to web apps that fetch many things at once. Sliding log is exact and memory-heavy (one entry per request). A sliding counter approximates it in O(1). Leaky bucket enforces a constant output rate, which suits protecting a fragile downstream. Fixed window is simplest but allows double the limit at the boundary.

Q35. Why did RefinePoint use different limiters at the gateway and in services?

Different jobs. At the gateway every request passes, so it needs O(1) memory per caller and burst tolerance: a token bucket holding a minute's requests that refills evenly, since a page load fires many calls. Inside services the limiter guards a few expensive or public actions, with low limits per hour or day where exact counting is worth a small timestamp deque: `SlidingWindowLimiter`.

Q36. Is a per-instance rate limiter wrong?

Not at small scale, if you understand what it gives you: with N instances a caller may get up to N times the configured limit, and uneven load balancing makes it less exact. It fails safe, costs no network hop, and has no external dependency. It is wrong for an exact contractual quota or when the instance count is large. Then use Redis with an atomic Lua script, and decide whether the limiter fails open if Redis is down.

Q37. If Redis goes down, should the rate limiter fail open or closed?

Usually open, with a local fallback limiter, because rejecting every customer due to a limiter dependency is a self-inflicted outage. Fail closed only where the limit protects something critical, such as a login or SMS endpoint exposed to abuse. Add a short timeout on the Redis call, so a slow Redis does not slow every request.

Q38. How do you warm a cache, and is it worth it?

Pre-load known hot keys at start-up or before a traffic peak, ideally from an access log, rate-limited so warming does not itself harm the DB. It is worthwhile for a big cold start or a predictable peak. Otherwise rely on lazy loading with stampede protection, plus a canary or gradual traffic shift so a new cache fills under partial load.

Q39. How would you size Redis and watch it in production?

Size from key count × average entry size plus overhead, with headroom for forks. Watch `used_memory`, `evicted_keys`, the hit ratio from `keyspace_hits/misses`, the slow log, connected clients, replication lag, and

latency from `--latency`. High `mem_fragmentation_ratio` suggests active defrag. An unexpected eviction rate on a non-eviction instance means writes are about to fail.

Q40. Spring Session with Redis vs JWT?

Redis sessions give instant revocation and a small cookie, but every request needs a lookup and Redis becomes critical. JWT is stateless and scales across services, but revocation is hard, so keep access tokens short-lived and use refresh tokens. RefinePoint uses Keycloak JWTs, verified locally by each service, and caches what the token does not carry (workspace roles) with a 30-second TTL.

Q41. Redis pub/sub vs Streams vs Kafka?

Pub/sub is transient: no history, no acknowledgement, slow subscribers get disconnected. Streams store entries with consumer groups, acknowledgement and replay, and suit moderate loads within Redis's memory. Kafka is a disk-based partitioned log with long retention and high throughput, for durable integration events. RefinePoint uses Kafka through an outbox because a lost audit or run event is not acceptable.

Q42. SQL vs NoSQL: how do you decide?

Start from access patterns and invariants. Choose SQL for multi-entity transactions, constraints and evolving queries; choose NoSQL when a specific pattern needs what relational systems do badly, such as huge write rates, nested variable documents, or relevance search. "NoSQL scales, SQL doesn't" is outdated: most companies never outgrow a well-indexed Postgres with partitioning and replicas. Reconciliation needs ACID and flexible reporting, so ours is relational.

Q43. What are the BASE properties?

Basically Available, Soft state, Eventually consistent — the counterpart to ACID, describing systems that favour availability and tolerate temporary inconsistency. Most modern stores are tunable rather than strictly one or the other, so ask about the specific consistency level per operation.

Q44. How does an LSM tree differ from a B+tree?

A B+tree updates pages in place, giving predictable reads but random writes. An LSM tree (Cassandra, RocksDB) appends to a log and memtable, flushes sorted immutable files and compacts them in the background, giving very fast writes but reads that may check several files, helped by Bloom filters. Write amplification comes from compaction in LSM and from page rewrites and WAL in B+trees.

Q45. How do you model one-to-many in MongoDB?

Embed if children are few, owned and read with the parent (order lines). Reference if they are many, shared, or updated independently (a customer's orders). For unbounded series use the bucket pattern (one document per hour of readings) because of the 16 MB limit and the cost of rewriting growing documents. Denormalise a few fields you display often and accept the update cost.

Q46. MongoDB vs a Postgres `jsonb` column?

`jsonb` with GIN indexes covers flexible attributes on otherwise relational data and keeps transactions and joins. Choose MongoDB when the whole model is document-shaped, you need its built-in horizontal sharding, or your teams want a schema-flexible workflow. Writing a `jsonb` value rewrites the entire TOASTed value, so very large, frequently updated documents are a poor fit.

Q47. How do Cassandra's consistency levels work?

Each read or write specifies how many replicas must respond: `ONE`, `QUORUM`, `ALL`, `LOCAL_QUORUM`. With replication factor N, `R + W > N` makes read and write sets overlap, so reads see the latest write. `QUORUM` / `QUORUM` with N = 3 is the usual balance. Conflicts are resolved last-write-wins by timestamp, so clock skew matters. Hinted handoff and read repair bring replicas back into line.

Q48. Why is a monotonically increasing partition key bad?

All new writes go to a single partition or node, which then becomes hot, whether in Cassandra, DynamoDB or a Mongo ranged shard key. Hash it, add a bucket component (`day + hash % N`), or use a compound key. The same logic drives choosing a tenant id over a timestamp when sharding.

Q49. How does Elasticsearch search work, and why not use it as the primary store?

An inverted index maps analysed terms to documents, and scoring (BM25) ranks results. It is near real time because new documents become searchable only after a refresh. It has no multi-document transactions, and mapping changes require a reindex. Therefore keep the truth in a database, feed the index through events or CDC, and be ready to rebuild it from scratch.

Q50. `text` vs `keyword` in Elasticsearch? What is deep-pagination's problem?

`text` is analysed for full-text search; `keyword` is stored as is for filters, sort and aggregations. `from + size` makes each shard gather `from + size` hits, so deep pages are expensive and limited by default to 10,000. Use `search_after` with a point-in-time view, similar to keyset pagination in SQL.

Q51. How would you add search to reconciliation breaks without Elasticsearch?

Start in Postgres: a `pg_trgm` GIN index for fuzzy text and `tsvector` for words, scoped by `org_id`. It stays transactional and needs no pipeline. Move to Elasticsearch only when relevance tuning, facets across large data, or query volume exceed what Postgres can serve — then feed it from the outbox.

Q52. What would make you add Redis to RefinePoint, concretely?

Multiple triggers: enough gateway instances that per-instance limits become meaningless; a contractual quota that must be exact; several instances all needing the same expensive computed view; or a role-revocation requirement stricter than a 30-second TTL can meet, which would need shared invalidation. Until then each new dependency is one more thing a solo developer must keep alive before launch.

Q53. Design a cache for a permission check hit on every request.

Keep it in-process, because a network hop per request defeats the purpose. Key on `org + user` with a TTL of seconds, bounded in size, negative results cached briefly. Decide and state the revocation delay; if it must be immediate, broadcast invalidations by Kafka or Redis pub/sub on top of the TTL. The serving path must not fail if the origin is briefly down, so decide whether stale answers are allowed and for how long. This is what `RemoteWorkspaceAccess` does, except it has no broadcast: 30 seconds is the stated bound.

Q54. Predict the behaviour: two `@Cacheable` methods call each other in one class.

```
@Service class S {
    @Cacheable("a") public int a(int x) { return b(x); }
    @Cacheable("b") public int b(int x) { return slow(x); }
}
```

Calling `s.a(1)` from outside caches under `a`. The inner `b(x)` is a self-invocation, so it bypasses the proxy and nothing is stored under `b`. A later external `s.b(1)` therefore runs `slow` again and only then caches it.

Q55. A cached list was mutated and now every user sees wrong data. Why?

In-process caches return references, so the caller's `list.add(...)` changed the cached instance for everyone. Redis hides this bug because each read deserialises a new copy, so it appears only when migrating to Caffeine. Cache immutable values (records with `List.copyOf`) or copy on read.

Q56. What happens to a Spring Boot app when Redis goes down?

By default cache operations throw and the request fails, so a cache outage becomes an application outage. Implement a `CacheErrorHandler` that logs the exception and treats it as a miss, set short command timeouts, and protect the database with a limiter because every request will now reach it. Test this path deliberately, with the cache actually stopped.

Q57. How do CDN and browser caching work with deploys?

Use content-hashed file names (`app.3fa9.js`) with `Cache-Control: public, max-age=31536000, immutable`, and short or no caching on the HTML that references them. A new deploy creates new URLs, so nothing needs purging. For API responses, `ETag` with `If-None-Match` saves bandwidth (304) but not the server's work unless the ETag is cheap to compute. Authenticated responses should be `private`.